



SPRINT 1 REPORT

COS40006 COMPUTING TECHNOLOGY PROJECT B

Prepared by Group 12:

Kenneth Hui Hong CHUA	102782494
Ivan Jia Wei HOAN	102786580
Kelvin Wen Kiong FONG	102782287
Nigel Zi Jun LING	102779140

Unit code: COS40006/EAT40006

Unit Name: Computing/ Engineering Technology Project B

Submission date: 24th March 2026

Contribution Summary (This Sprint).....	1
Contribution Details (This Sprint)	2
1 Sprint plan	18
1.1 Sprint Goal.....	18
1.2 Scope.....	18
1.3 Assumptions & Dependencies	19
1.4 Capacity & Forecast	20
1.5 Definition of Done.....	20
1.6 Sprint Backlog	21
1.7 Sprint milestone & Review artifacts.....	22
2 Risk Management Plan	23
3 Sprint progress	27
3.1 Sprint Deliverables	28
3.2 Repository and workflow evidence	52
3.2.1 GitHub.....	52
3.2.2 WhatsApp Chat.....	53
3.2.3 Microsoft Teams.....	54
3.2.4 Linear	55
4 Sprint review	59
4.1 Demonstration Session and Client Communication	59
4.2 Feedback Received.....	61
4.3 Critical Analysis of Progress	62
5 Retrospect (Critical review of the process)	63
6 Lessons learned (Critical review of SPRINT 1 experience and future plan).....	67

Contribution Summary (This Sprint)

	Contribution	Initiatives	Communication					Respect
Team Member	Finished tasks in time or on time with an acceptable quality	Self-assigned tasks and proactively found solutions to problems	Attended all team meetings	Attended all supervisor meetings	Attended all client meetings	Replied every time within an acceptable timeframe	Kept the team updated about the status	The student respected others and listened to different opinions
Kenneth Hui Hong CHUA	Yes, in case of can't make on time will inform the team	Self-researching, find out problem, provide solution to the team	Yes	Yes	Yes	Yes	As leader of team, always updating the status	Yes, listening to team member opinion
Ivan Jia Wei HOAN	If cannot finish will try adjust the task	Will find problem and solution for the project	Yes	Yes	Yes	Reply as soon as possible	Will notice if have any changes	Try to discuss with others when different opinion
Kelvin Wen Kiong FONG	Built the foundation of the system and made the webpage responsive	Helped with tasks when needed and tried my best to solve problems	Yes	Yes	Yes	Reply ASAP	Regularly informed the team about our progress and any changes in the project	Talk with all team members to share feedback and check on any updates or changes
Nigel Zi Jun LING	Follow the requirement and finish task as soon as possible with acceptable quality	Help teams to solve	Yes	Yes	Yes	Reply as soon as possible	Notice every change in physical meetings and WhatsApp group	Communicate with every member to exchange feedback and follow up changes

Contribution Details (This Sprint)

In this section, every member of the team should write a summary of what they have contributed during this sprint. All contributions must be supported by evidence, such as the number of GitHub commits, a screenshot of developed screens, a summary/outcome of research, etc.

Kenneth Hui Hong CHUA

During Sprint 1, my contributions were focused on building and refining the AI powered book recommendation system from the ground up. I completed 11 issues on Linear and contributed approximately 15 to 20 hours to the sprint.

1. AI Recommendation System — Architecture & Setup

I built the full backend architecture for the AI book recommendation system, integrating Google Gemini 2.5 Flash as the large language model. For the initial setup and testing phase, I used a personal Google Cloud API key to validate the connection and accelerate development. I also designed a Retrieval Augmented Generation pattern to ensure the AI only recommends books that are actually in the library catalog. The frontend chat interface was then created and added to the navigation bar so users can access the feature directly.

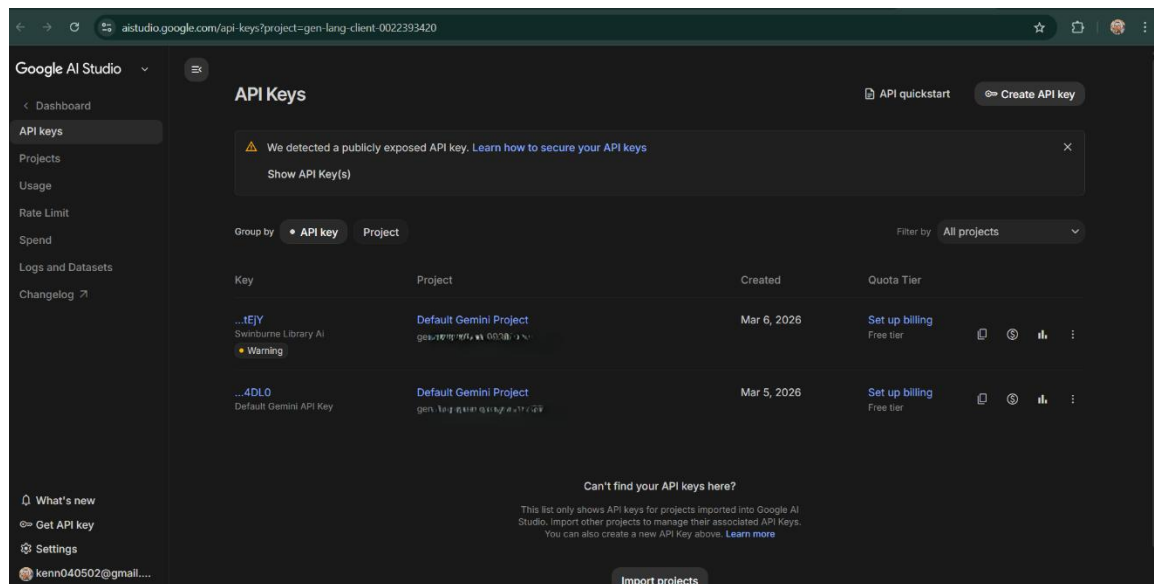


Figure 1: AI recommendation chat interface

2. AI Model Refinement & Optimization

After the initial build, I focused on improving the quality of the recommendations. I refined the system prompt to make the AI respond more accurately to user requests, and improved the keyword extraction logic to better match user descriptions with book tags. I also added greeting detection so the AI can respond naturally to casual messages from users.

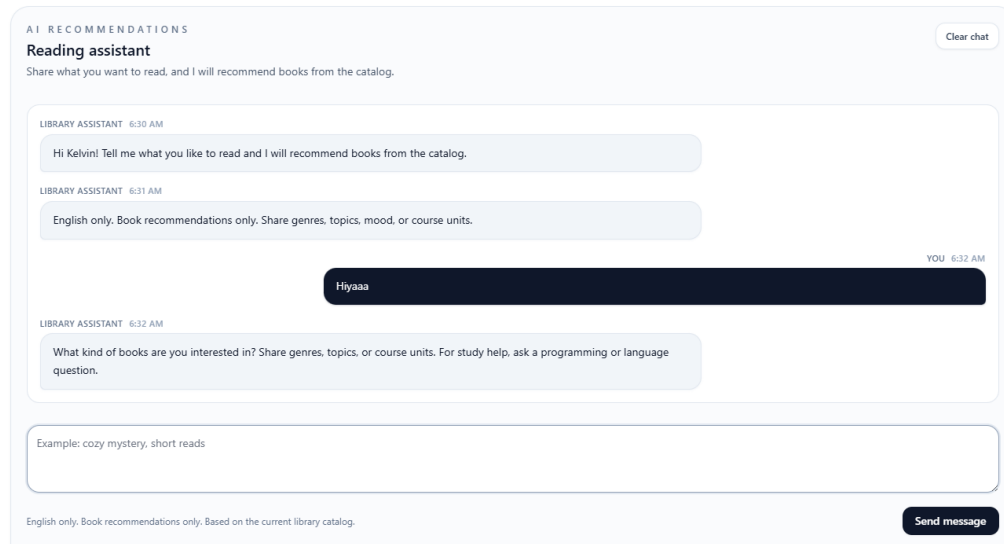


Figure 2: AI chat showing recommendations

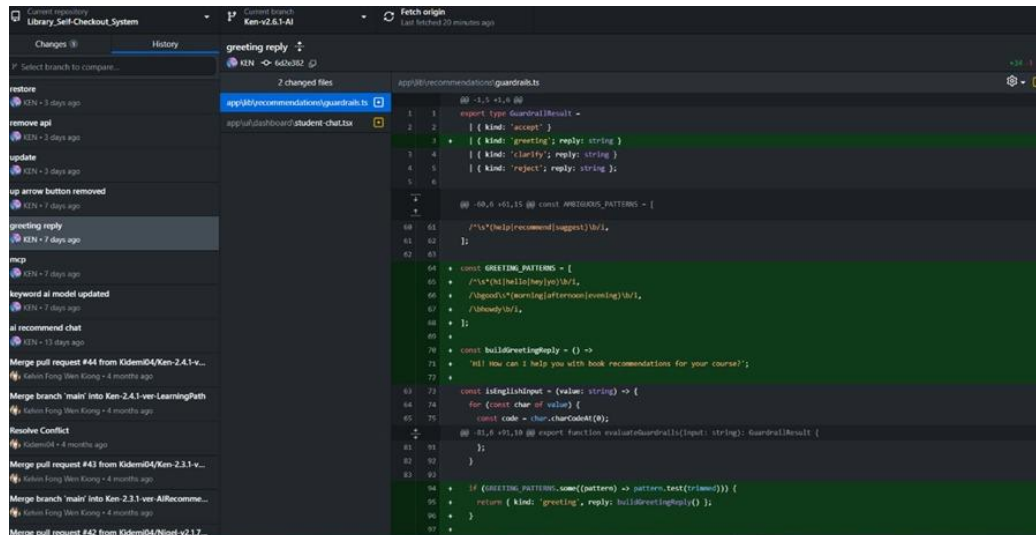


Figure 3: GitHub commit showing greeting detection logic added to the AI guardrails

3. Content Safety & Guardrails

I implemented content restrictions to ensure the AI only responds to library related topics. Any off topic queries are intercepted before they reach the model. I also removed a temporary development API endpoint to make sure no test routes were included in the production build.

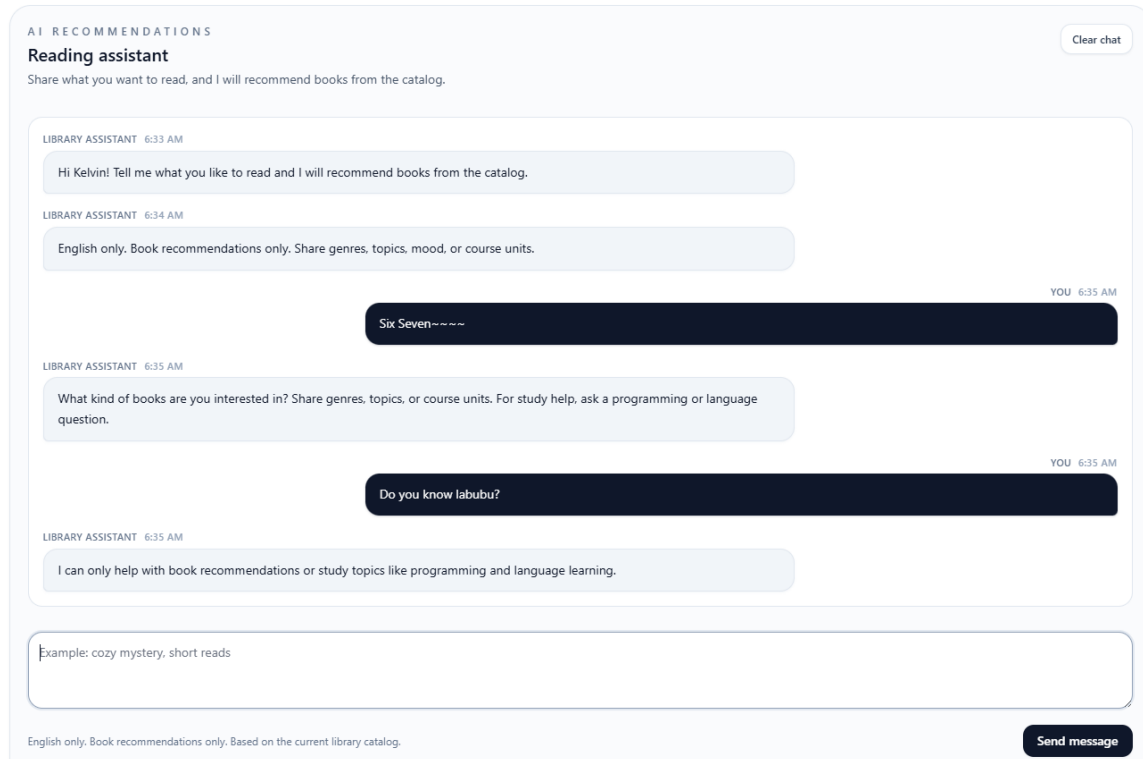


Figure 4: AI rejecting off-topic query

4. Identified Challenges & Next Steps

During testing, I identified two key limitations in the current implementation. The first is that the AI struggles with multi-topic queries, where a request combining two subjects may only return results for one of them. The second is that the system does not yet correctly handle negative instructions, such as when a user asks to avoid a specific type of book. Resolving these issues is a priority for the next sprint, with a focus on improving prompt design and user intent processing.

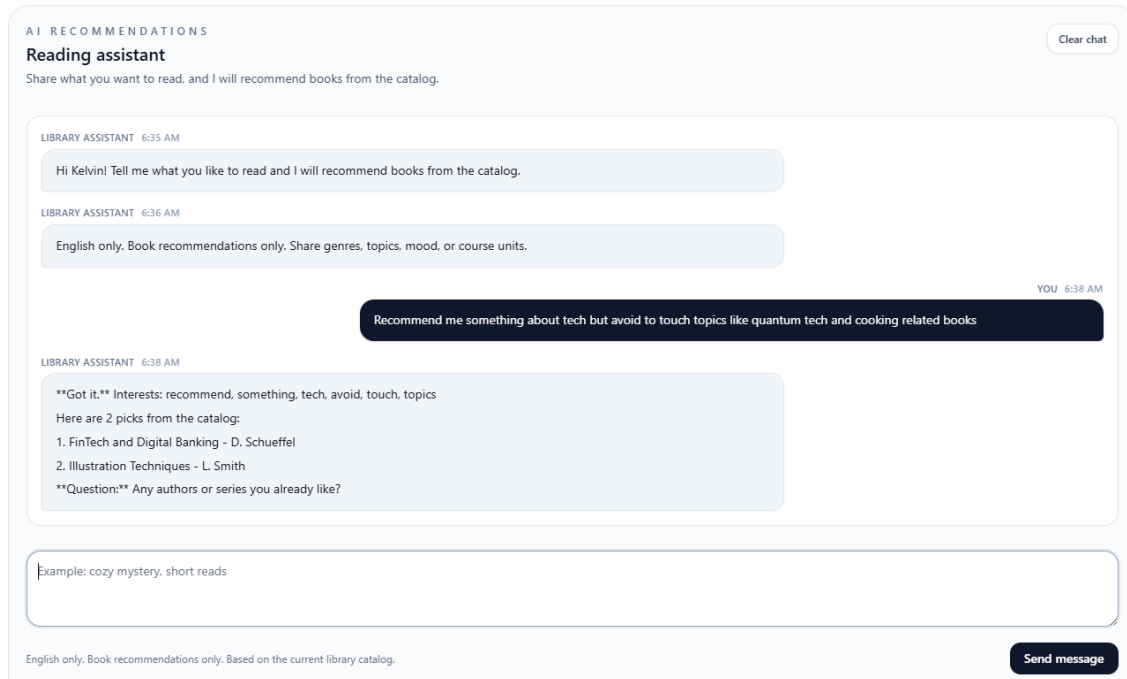


Figure 5: example of AI limitation

Kelvin Weng Kiong FONG

During Sprint 1, my contributions spanned four key areas: system deployment and CI/CD pipeline stability, branch integration, user experience enhancements, and testing infrastructure. I completed 12 issues on Linear and contributed approximately 10 to 15 hours to the sprint.

1. System Deployment & CI/CD Pipeline

I was in charge of setting up the team's production deployment on Vercel. During the sprint, I ran into two deployment blockers and fixed both of them. One was a lockfile issue, and the other was a security vulnerability in Next.js that Vercel flagged automatically. I also upgraded the Supabase dependency to remove a recurring build warning. After resolving these, the website was successfully deployed and running in production.

Library Self-Checkup System

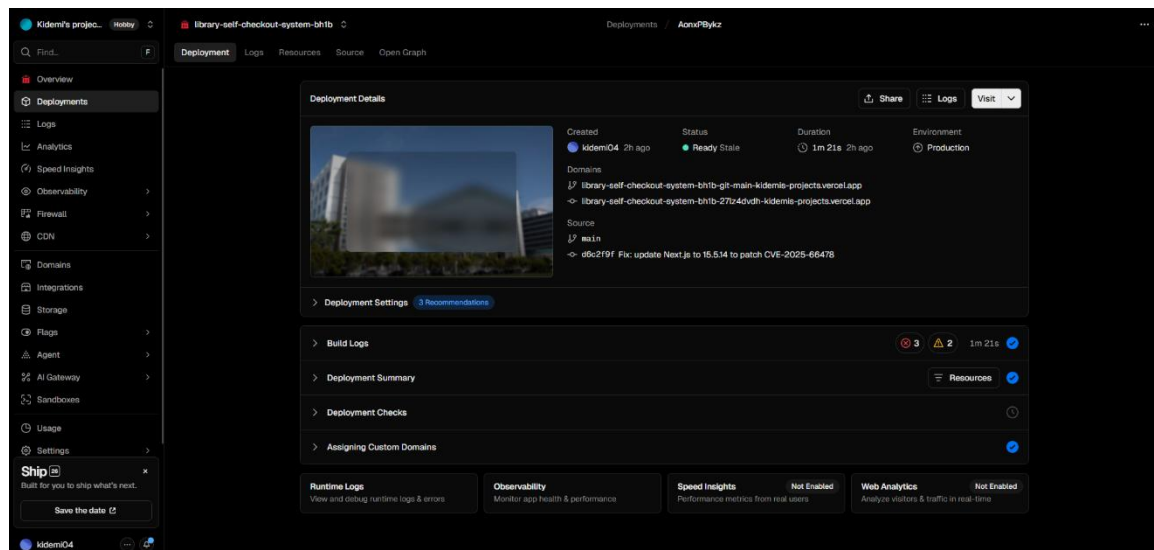


Figure 6: Vercel Deployment — Status: Ready

2. Branch Integration & Conflict Resolution

I was responsible for merging all feature branches from the team into the main codebase. There were six branches in total from all team members, and each merge required resolving conflicts across several critical files. I also verified that the build remained stable after every merge. This work allowed the team to develop features independently without breaking the shared codebase.

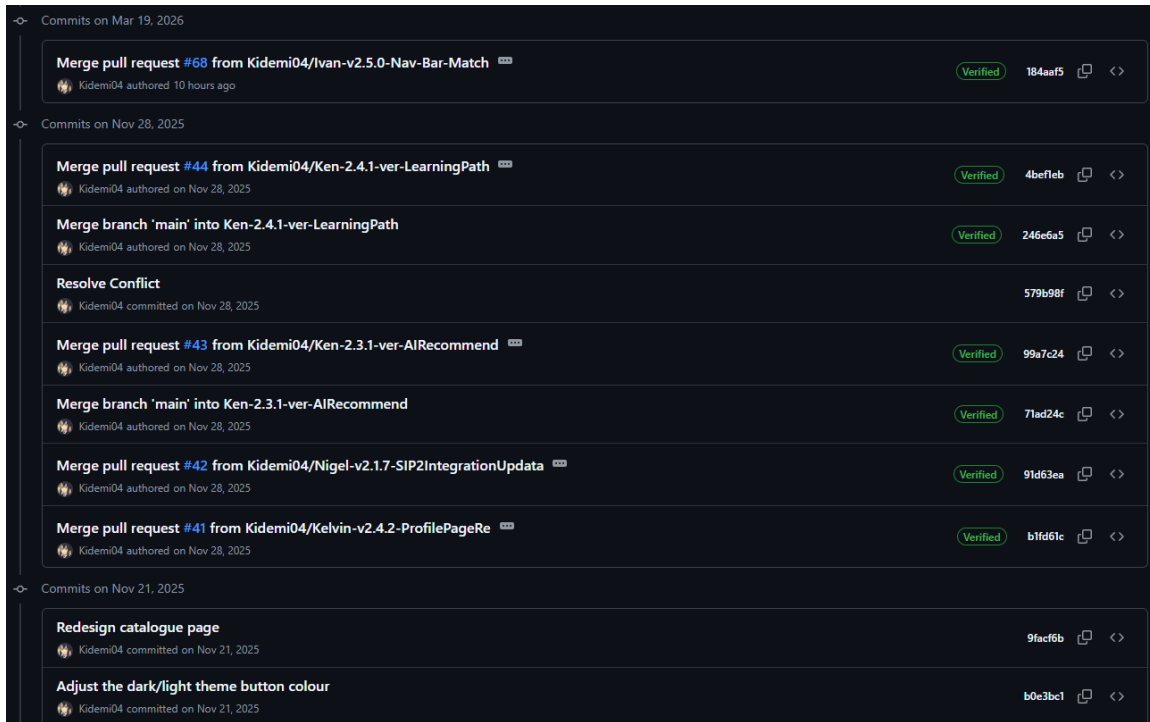


Figure 7: GitHub commit history showing merge pull requests

3. User Experience Enhancements

I implemented three frontend improvements to make the system feel more polished and reliable. The first was a page loading skeleton with shimmer animation, which removes the blank flash users see while a page is loading. The second was error boundary pages that show a friendly message when something goes wrong instead of a broken screen. The third was a custom 404 page that guides users back to the main pages of the system.

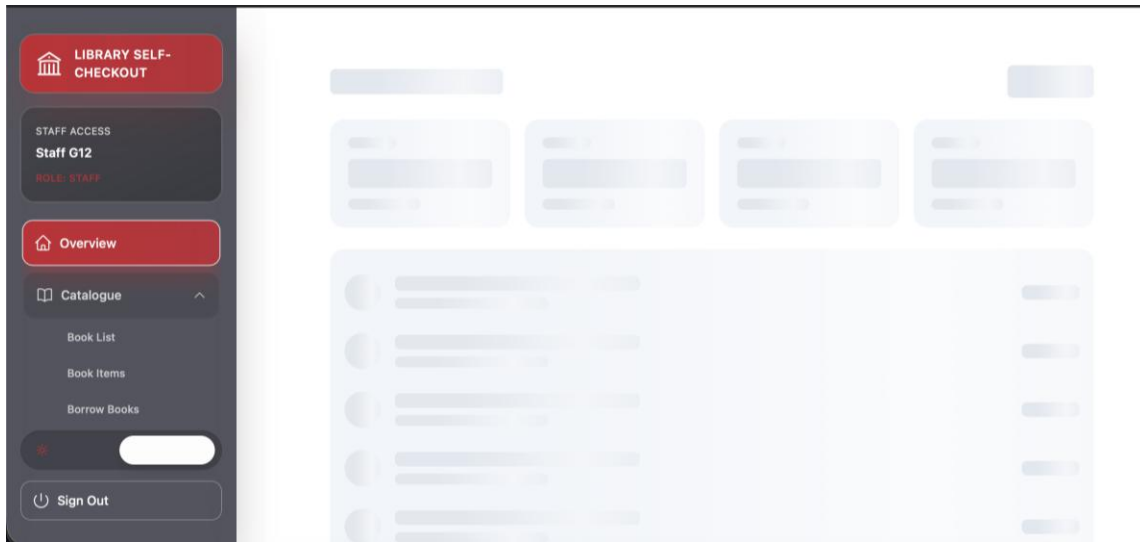


Figure 8: Loading Screen

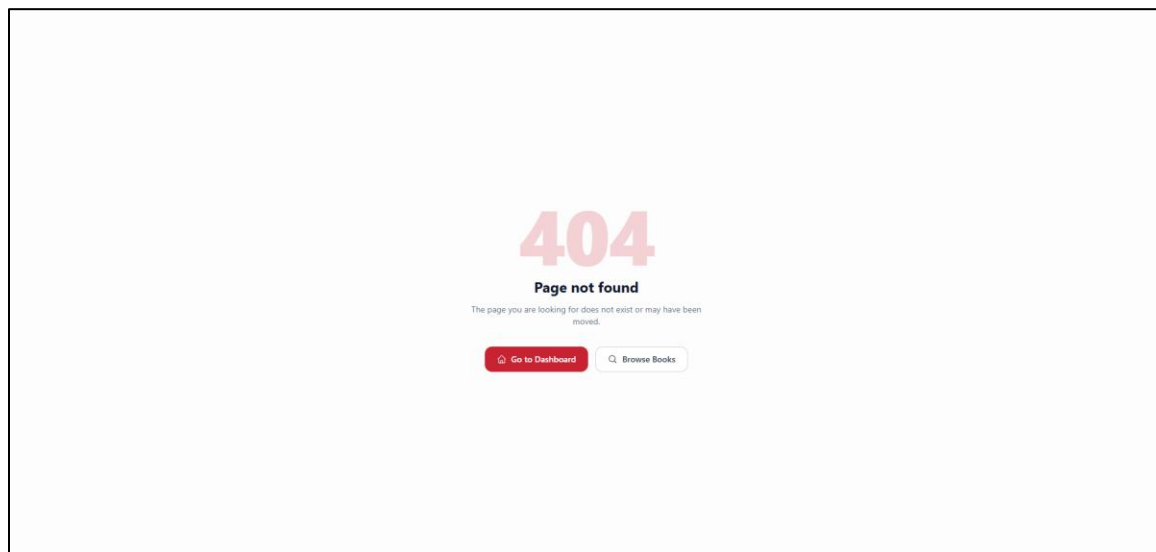


Figure 9: Custom 404 page

4. Testing Infrastructure

Building on the Jest testing framework set up by Ivan, I wrote unit tests for three UI components, covering a total of 20 test cases across rendering, user interaction, and accessibility. I also fixed two existing test failures that were caused by a file rename done by another team member. These contributions brought the total test suite to 28 passing tests with zero failures.

```
at process.env.NODE_ENV.exports.act (node_modules/.pnpm/react@19.1.0/node_modules/
at node_modules/.pnpm/@testing-library+react@16.3_06bcd1a012c3501633424e888
at renderRoot (node_modules/.pnpm/@testing-library+react@16.3_06bcd1a012c3501633424e888
at render (node_modules/.pnpm/@testing-library+react@16.3_06bcd1a012c3501633424e888
at Object.<anonymous> (__tests__/login_client.test.jsx:29:9)

PASS __tests__/sign_out_button.test.jsx
PASS __tests__/notification_filter.test.jsx
PASS __tests__/home_page.test.jsx

Test Suites: 6 passed, 6 total
Tests: 28 passed, 28 total
Snapshots: 0 total
Time: 5.749 s
Ran all test suites.
PS C:\Users\fongk\Documents\Assignment\Library_Self-Checkout_System>
```

Figure 10: Terminal showing 28 tests passing

Nigel Zi Jun LING

During Sprint 1, my contributions were focused on three key areas: security hardening, learning path implementation, and AI recommendation system debugging. I contributed approximately 10 to 15 hours to the sprint.

1. Learning Path Implementation

I implemented the initial learning path feature, which generates a structured reading progression from beginner to advanced level based on the user's selected topic. The learning path is powered by the Gemini AI model, which intelligently classifies books from the library catalog into Beginner, Intermediate, and Advanced stages based on the book's title and tags. Users can either type a custom topic or select from preset quick-topic chips. Each book card displays the title, author, a short reason why it fits that level, and real-time availability status pulled directly from the library database. Book availability is fetched live from Supabase so users always see accurate availability status rather than hardcoded data. I also built a fallback algorithm using an even split approach to ensure the feature continues to work even if the AI service becomes unavailable, so the system will never break. However, there are still some of the limitations, the system cannot perfectly show the book availability, some of the books that shown on loan are actually available, also right now users cannot save a learning path or mark stages as completed, so still need adjustment and improvement.

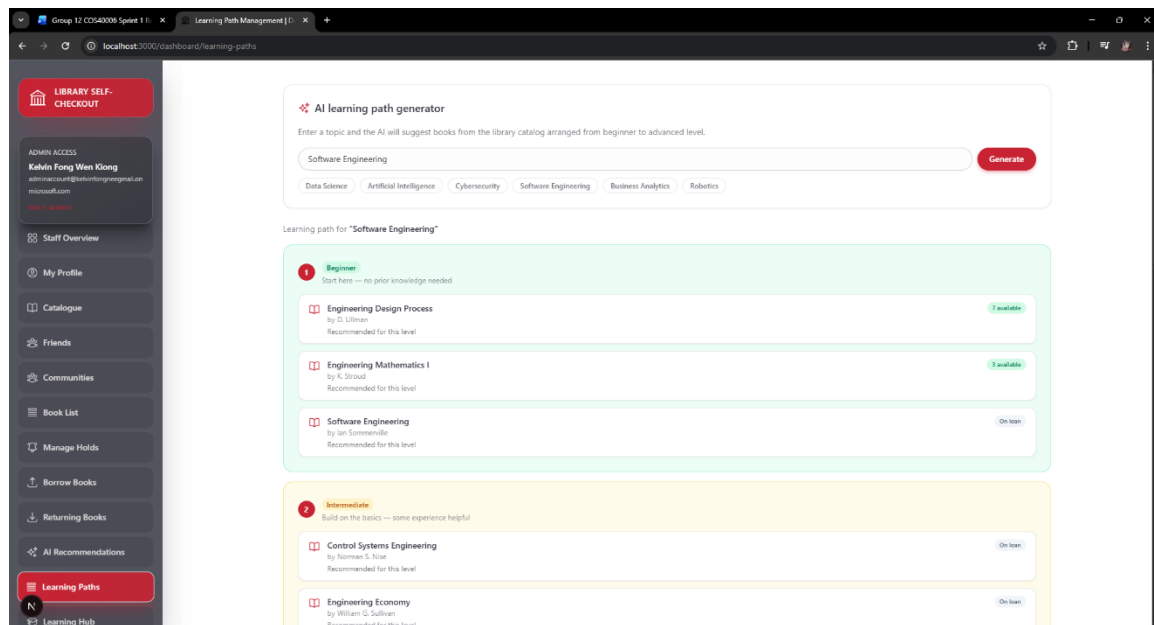


Figure 11: Learning path page showing Beginner level

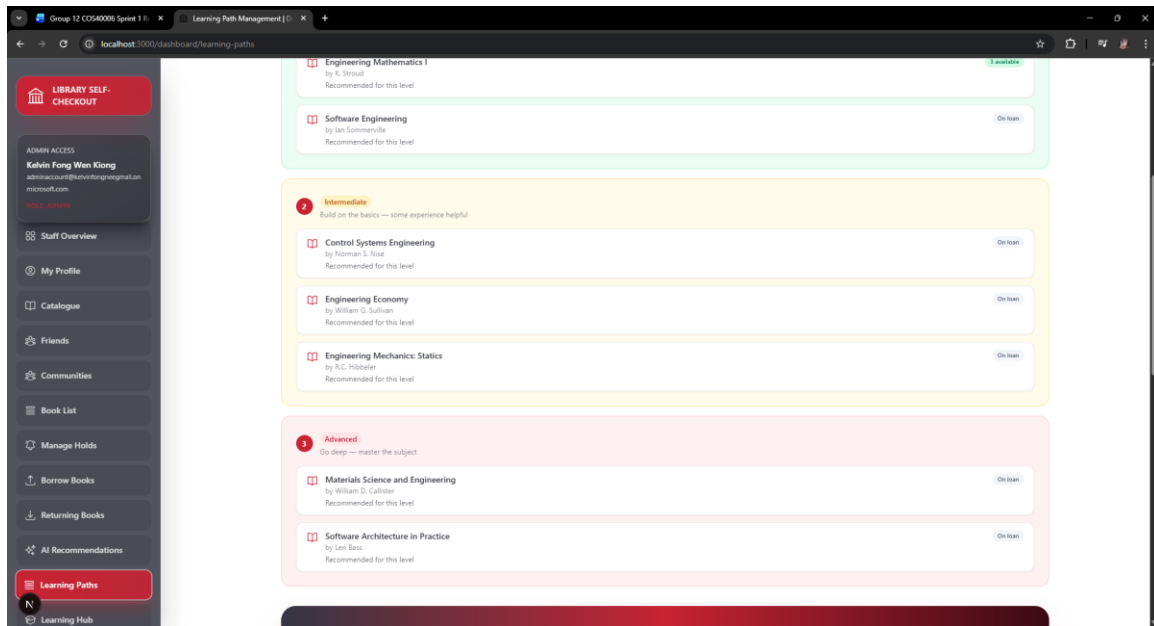


Figure 12: Learning path page showing Intermediate and Advanced levels

2. Security Hardening

2.1 SIP2 Communication & Credential Protection

I identified and resolved a critical security vulnerability where sensitive credentials were being exposed in browser logs and network requests. To fix this, I implemented server side handling for all SIP2 communication, ensuring that sensitive operations such as check in and check out are processed entirely on the server and never exposed to the client. I also verified that all environment variables are inaccessible from the browser, and confirmed that no sensitive keys appear in network payloads.

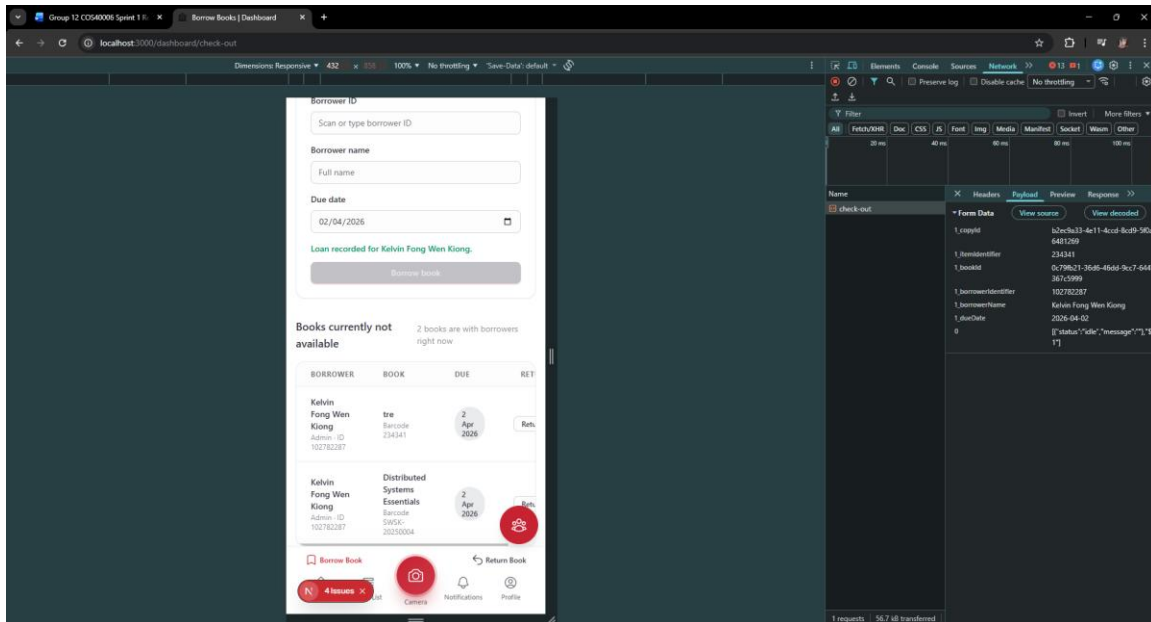


Figure 13: Network payload showing no sensitive credentials exposed

2.2 Authentication & Access Control

I implemented centralized session validation and role based access control to restrict admin and user access to appropriate routes, and prevented unauthorized access through server side redirects. Although borrower information is still included in some payloads, a further improvement has been identified where only the borrower identifier should be sent and additional details retrieved server side to prevent client side data manipulation.

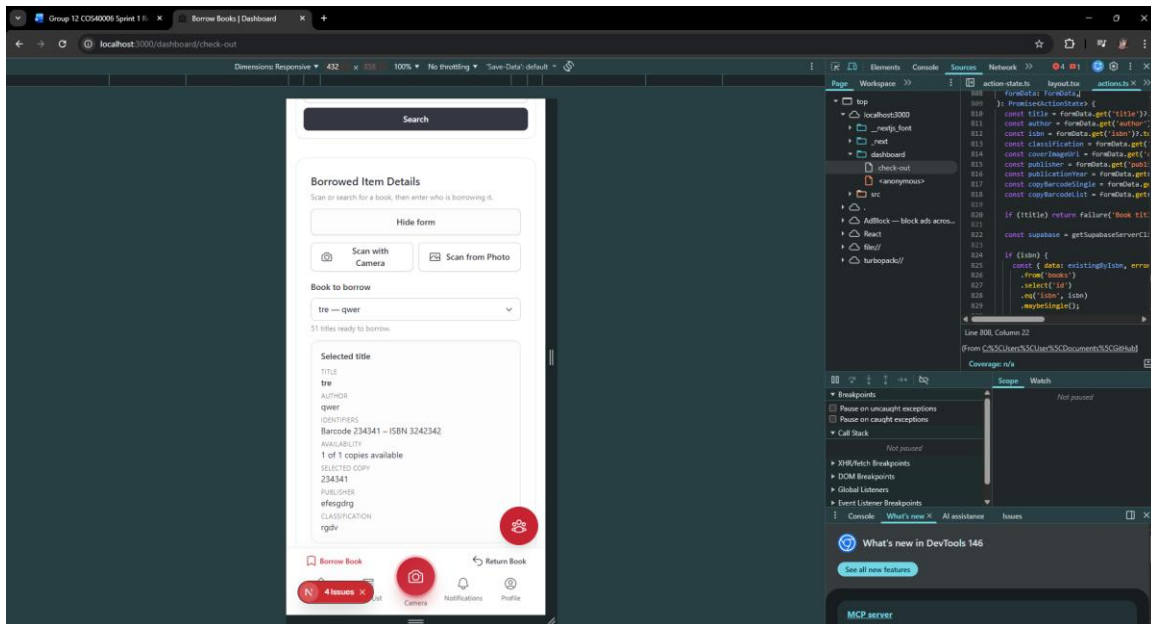


Figure 14: Server side code handling sensitive operations

3. AI Recommendation System Debugging

I assisted in investigating and fixing issues with the AI recommendation system, working alongside Kenneth to improve the accuracy of the AI responses. This included identifying and resolving problems with how the system handles certain user queries, particularly those involving negative constraints such as when a user asks the AI to avoid recommending a specific type of book.

Ivan Jia Wei HOAN

During Sprint 1, my contributions were focused on four key areas: UI fixes and improvements, code standardisation, component development, and testing infrastructure. I completed 18 issues on Linear and contributed approximately 15 to 20 hours to the sprint.

1. UI Fixes & Improvements

I addressed several interface issues to enhance user experience and improve responsiveness across the system. This included refactoring the mobile navigation bar to fix broken route paths and improve the hamburger menu layout, resolving a white flash issue on the login page in dark mode by restructuring the theme provider logic, updating the login page icon to match the overall visual design, and making minor alignment and spacing adjustments to the admin page and dashboard title bar.

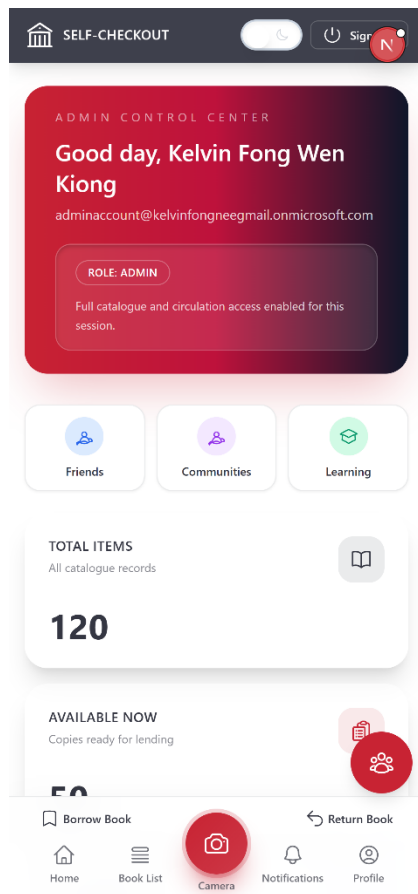


Figure 15: Unfixed mobile nav bar

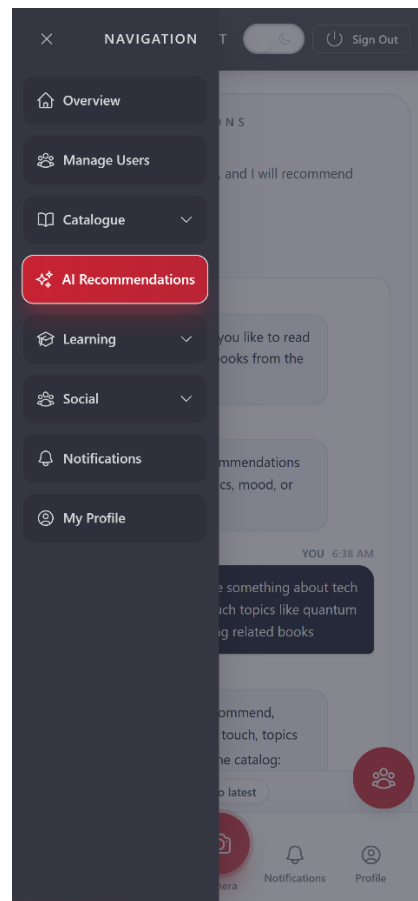
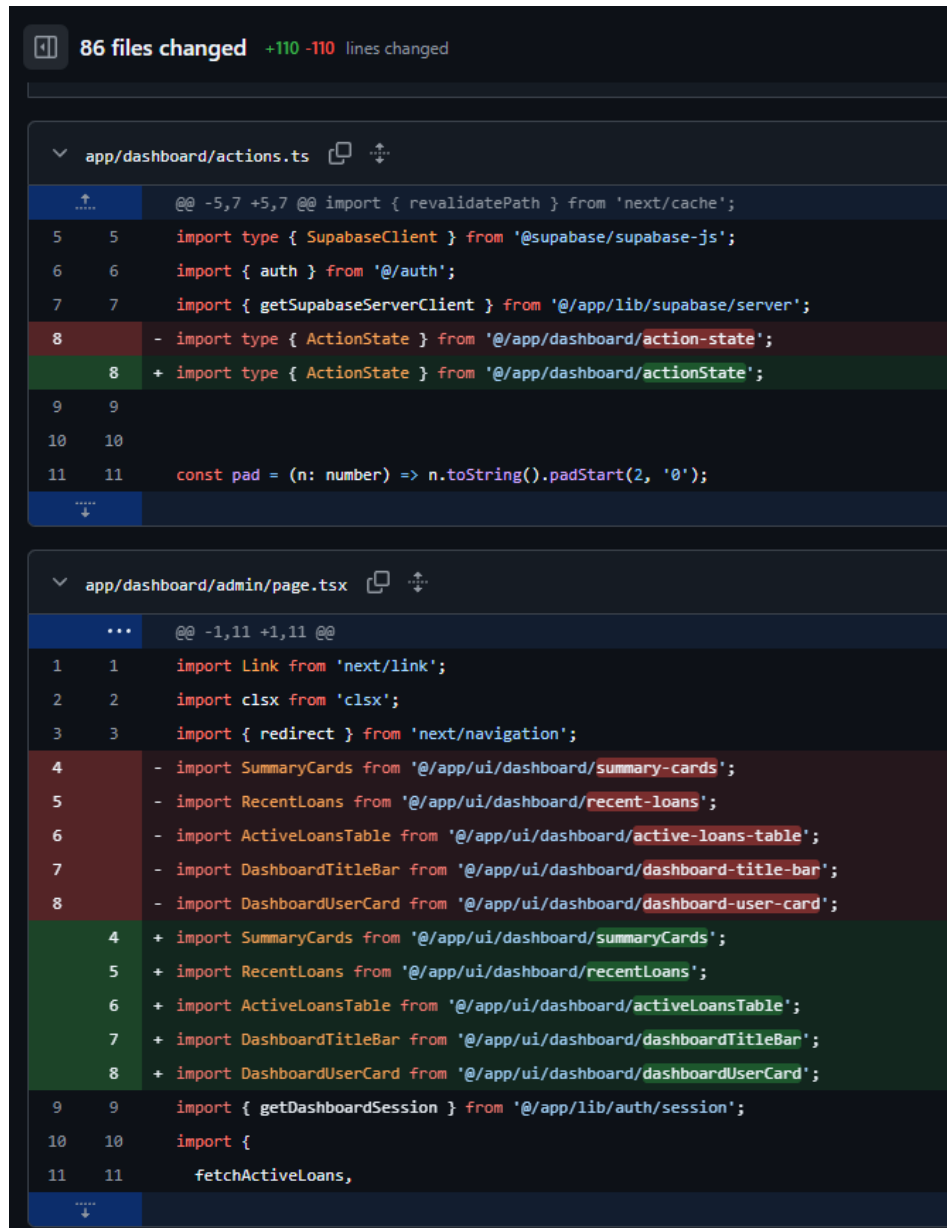


Figure 16: Fixed mobile nav bar

2. Code Standardisation

I was responsible for standardising the codebase to meet the client's formatting requirements. This involved renaming all file and folder names to camelCase across the project, and migrating all Supabase table name references in the query layer from snake_case to PascalCase across a total of 45 query calls in two passes. I also removed the redundant custom register page as user registration is now fully handled by Clerk.



The screenshot displays a code editor interface with two files open, showing the process of standardizing Supabase table names from snake_case to PascalCase. The top file, `app/dashboard/actions.ts`, shows a diff where line 8 is updated from `import type { ActionState } from '@app/dashboard/action-state';` to `import type { ActionState } from '@app/dashboard/actionState';`. The bottom file, `app/dashboard/admin/page.tsx`, shows a diff where lines 4 through 8 are updated from snake_case to PascalCase: `SummaryCards`, `RecentLoans`, `ActiveLoansTable`, `DashboardTitleBar`, and `DashboardUserCard`. The editor interface includes a file explorer on the left, a diff view with line numbers, and a search bar at the top.

```
86 files changed +110 -110 lines changed

app/dashboard/actions.ts
@@ -5,7 +5,7 @@ import { revalidatePath } from 'next/cache';
5 5 import type { SupabaseClient } from '@supabase/supabase-js';
6 6 import { auth } from '@auth';
7 7 import { getSupabaseServerClient } from '@app/lib/supabase/server';
8 - import type { ActionState } from '@app/dashboard/action-state';
8 + import type { ActionState } from '@app/dashboard/actionState';
9 9
10 10
11 11 const pad = (n: number) => n.toString().padStart(2, '0');

app/dashboard/admin/page.tsx
@@ -1,11 +1,11 @@
1 1 import Link from 'next/link';
2 2 import clsx from 'clsx';
3 3 import { redirect } from 'next/navigation';
4 - import SummaryCards from '@app/ui/dashboard/summary-cards';
5 - import RecentLoans from '@app/ui/dashboard/recent-loans';
6 - import ActiveLoansTable from '@app/ui/dashboard/active-loans-table';
7 - import DashboardTitleBar from '@app/ui/dashboard/dashboard-title-bar';
8 - import DashboardUserCard from '@app/ui/dashboard/dashboard-user-card';
4 + import SummaryCards from '@app/ui/dashboard/summaryCards';
5 + import RecentLoans from '@app/ui/dashboard/recentLoans';
6 + import ActiveLoansTable from '@app/ui/dashboard/activeLoansTable';
7 + import DashboardTitleBar from '@app/ui/dashboard/dashboardTitleBar';
8 + import DashboardUserCard from '@app/ui/dashboard/dashboardUserCard';
9 9 import { getDashboardSession } from '@app/lib/auth/session';
10 10 import {
11 11 fetchActiveLoans,
```

Figure 17: file structure or Supabase query update

3. Component Development

I developed several reusable UI components to improve consistency and reduce duplicated code across the project. This included building a shared TabSwitch component that replaced approximately 130 lines of duplicated code across two pages, creating a Notification Filter component to support category based filtering, and integrating the AI recommendations chat interface into the overall page layout.

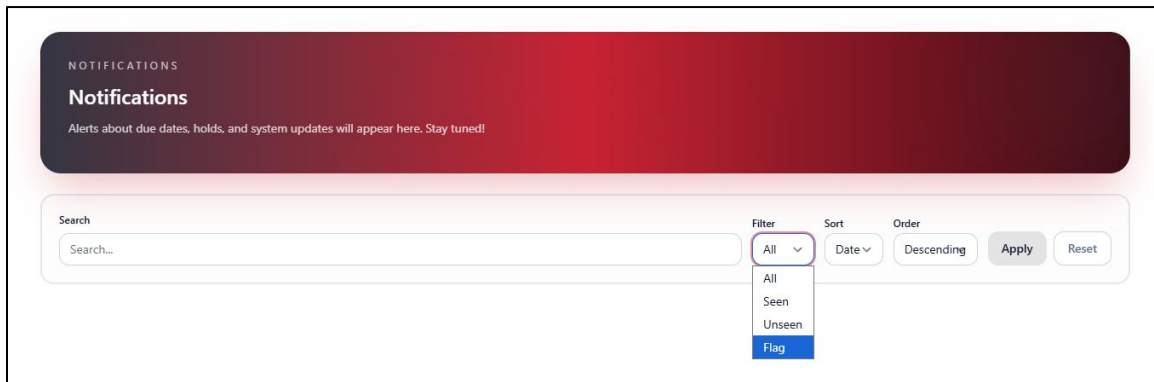


Figure 18: Notification Filter UI

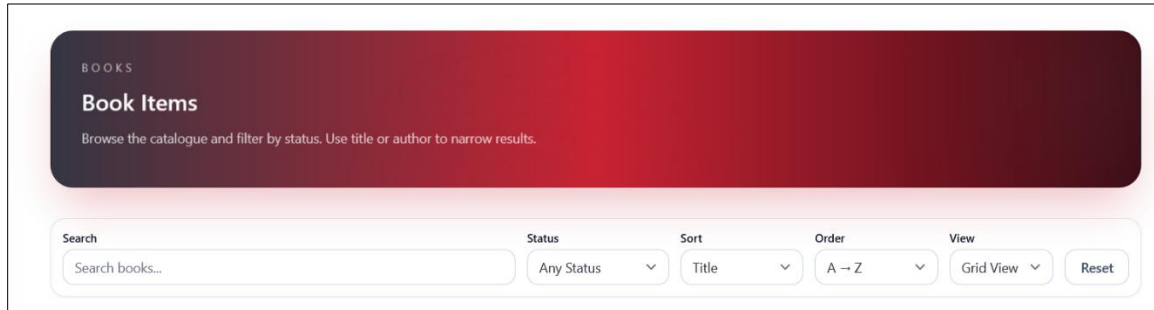


Figure 19: Book Search Filter

4. Testing Infrastructure

I set up the project's testing infrastructure from the ground up. This involved selecting Jest as the testing framework based on the client's recommendation, configuring it within the Next.js environment, and integrating it into a GitHub Actions workflow so that tests run automatically on every push. I also wrote unit tests for the login page covering core scenarios such as form rendering, input validation, and submission behaviour. I also updated outdated package dependencies identified through an npm audit.

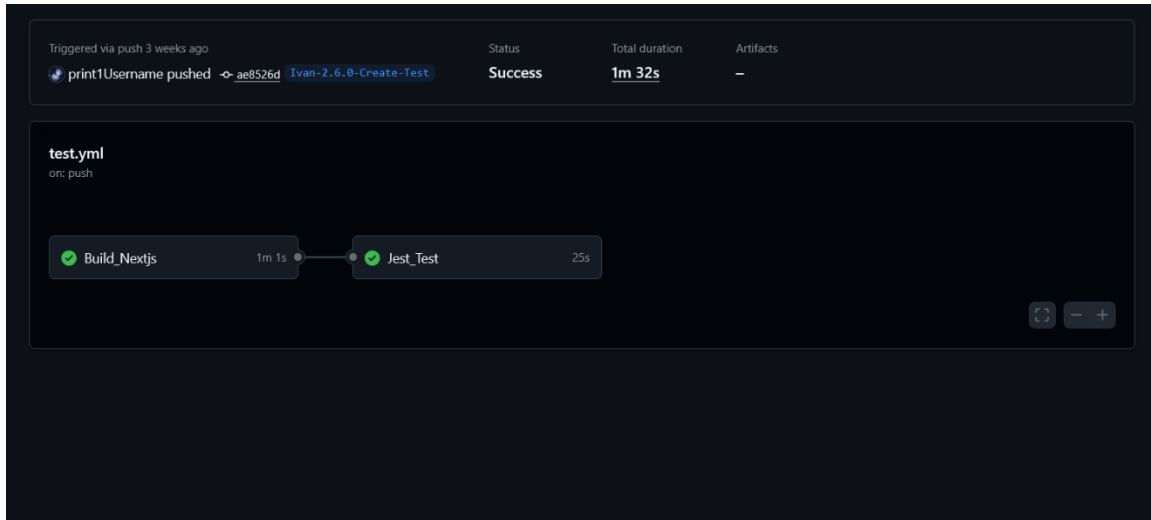


Figure 20: GitHub Actions workflow showing Build and Jest steps succeeding

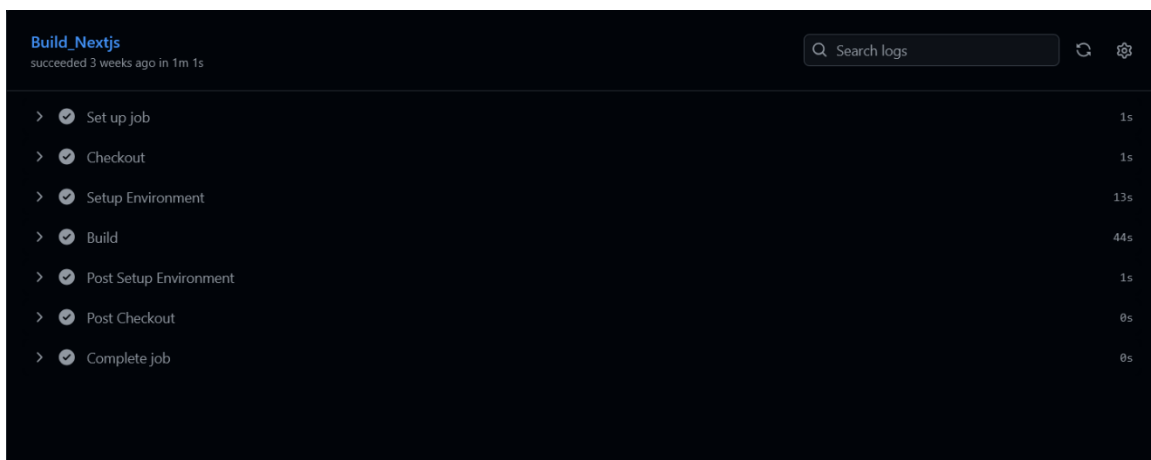


Figure 21: Build_Nextjs job details showing all steps passed

1 Sprint plan

1.1 SPRINT GOAL

In Sprint 1, we aim to add enhancements to our system building upon the foundation established in the previous semester. Following the client's feedback from Sprint 3, which highlighted the need to address security vulnerabilities and implement AI powered features, this sprint focuses on delivering an AI book recommendation system to help users pick books based on their personal interests, and a learning path system to build a structured learning programme with books tailored to the user's interest and level. Furthermore, we planned to integrate a LinkedIn Learning system that lets users search and browse online courses by topic, which could be connected to the AI book chat so that one query returns both library books and LinkedIn courses together as a complete learning package. We also addressed outstanding issues from the previous semester, including sensitive log message exposure and other cybersecurity vulnerabilities identified during the Sprint 3 code review session.

1.2 SCOPE

The scope of Sprint 1 included implementing the basic foundation for the AI recommendations system, learning path system, and LinkedIn Learning integration to make the system more diverse with additional features. Website security fixes and API adjustments were also planned to be completed in this sprint. All basic and enhancement features are planned to be fully completed and deployed in the next sprint.

1.3 ASSUMPTIONS & DEPENDENCIES

To ensure the successful delivery of Sprint 1, we are proceeding based on the following key assumptions and have identified primary dependencies:

Assumption:

The third-party Google Gemini and LinkedIn Learning APIs will be consistently available and perform as documented. The core application infrastructure and Supabase database are assumed to be stable enough to support new features without requiring major architectural changes. The development and testing environments are also assumed to accurately reflect production conditions. Finally, client access to a LinkedIn Learning subscription with administrative rights is presumed for configuration and testing.

Dependencies:

The refinement of new features, particularly the Learning Path management flow, depends on receiving timely client feedback after the sprint review. The implementation of the LinkedIn Learning integration requires the receipt of API credentials from the client's IT department. User Acceptance Testing (UAT) for the RFID detection gate is dependent on the availability of the physical hardware for on-site testing. Project security and functionality also remain dependent on open-source package maintainers to provide timely patches for any critical vulnerabilities

1.4 CAPACITY & FORECAST

Team Member	Estimated Hours
Kenneth Hui Hong CHUA	15 to 20 hours
Ivan Jia Wei HOAN	15 to 20 hours
Kelvin Wen Kiong FONG	15 to 20 hours
Nigel Zi Jun LING	15 to 20 hours
Total	About 100 hours

*Table 1: Sprint 1 covers Week 1 to Week 3 with four active members.
The table below shows the estimated time contribution per member*

We spent most of our time fixing existing issues and implementing new features. Approximately 20 hours were dedicated to researching AI system integration and LinkedIn Learning to gain the necessary knowledge for implementation. Security related issues such as sensitive message exposure were prioritised in this sprint as they posed a serious cybersecurity risk. We forecast meeting the sprint goal by delivering the basic functionality of all planned features to support future progress and performance.

1.5 DEFINITION OF DONE

A story is considered done when the website is able to perform the basic functions of the planned enhancement features and the identified cybersecurity issues have been resolved. For the AI recommendation system, it should successfully integrate an AI into the system through an API key and be able to perform basic functions such as detecting keywords from the user's prompt and recommending books with related topics. The learning path system should be able to generate a structured path from beginner to expert with relevant library books recommended at each stage. Furthermore, the website should have a simple Jest test created and deployed via GitHub Actions. The critical React security vulnerability identified during this sprint should also be patched and verified.

1.6 SPRINT BACKLOG

The sprint backlog covers AI recommendation integration, learning path integration, LinkedIn learning integration, and other bug fixing.

ID	Item	Acceptance criteria	Est.hrs.	Assignee
S1-01	Website Deployment & CI/CD Pipeline	The website is successfully deployed on Vercel and accessible via a public URL. All deployment blockers including dependency and security issues are resolved. The production build completes without errors or warnings.	15	Kelvin
S1-02	AI-Task: Integrate Google Gemini API	API key is securely stored as an environment variable. A dedicated service module is created to handle API requests. A test function can successfully send a prompt and receive a valid response. Errors from the API are handled gracefully.	2	Kenneth
S1-03	API emulator testing for sensitive message showing	Testing CheckIn and CheckOut to make sure sensitive messages like sip terminal password, patron pass word, and information like API keys only show in the API emulator side.	3	Nigel
S1-04	Implement core RAG pipeline for recommendations	User's prompt is processed by the backend. Keywords are extracted and used to query the Supabase book catalog. Retrieved book candidates are sent to the Gemini API for ranking. The final list of recommendations is returned to the front end.	2	Kenneth
S1-05	implement initial safety guardrails	An anti-spam rate limit is active and returns an error if triggered. A language check rejects non-English input with a specific message.	2	Kenneth

S1-06	Investigate and document AI response issues	The issue with multi-topic queries (e.g., "AI and finance") is reproduced and documented. The issue with negative prompts (e.g., "don't recommend coding") is reproduced and documented. An initial plan for resolving these issues has been created.	3	Kenneth, Nigel
S1-07	Learning path basic implementation	Learning path can generate lists of books from beginner to advance level based on selected topic and shows the books status such as available or on loan	2	Nigel
S1-08	React Major Bug Fixed	Search for and learn about React bugs, identify and fix the issues	10	Ivan
S1-09	Optimize and improve UI	Continuously optimise and standardise the user interface across desktop and mobile platforms	15	Ivan
S1-10	Fixing UI bug	Identify and fix any UI bugs that are discovered	10	Ivan
S1-11	Create Jest test and GitHub Action	Researching how to use GitHub Actions and integrating with Jest	8	Ivan

1.7 SPRINT MILESTONE & REVIEW ARTIFACTS

In the first week of Sprint 1, our main focus was on continuously refining additional features, such as AI powered recommendations and learning path management. Additionally, this sprint served as the foundation for future deployment and testing phases, ensuring that all new features are built on a stable and secure codebase. The daily communication tool of choice was WhatsApp, while meetings were conducted via Microsoft Teams. This section also shows the evidence of the development process, as indicated by the presence of GitHub commit history, screenshots of all the interfaces created, as well as test logs for backend integration validation.

2 Risk Management Plan

As part of our project workflow, we keep a risk register that we revisit and update at the beginning of every sprint. Each risk is looked at in terms of how likely it is to happen and how badly it could affect the project, and we assign someone on the team to own it. The register covers risks across the whole project, not just the current sprint.

We have updated the register below based on what we learned in COS40005 Sprint 3. Some risks from last semester have been resolved or reduced, and we have added new ones that came up during Sprint 1.

No.	Risk	Likelihood	Impact	Rating	Responsible	Mitigation & Current Status
1.	Progress delay due to uneven skills and procrastination	High	High	Medium	Kelvin Fong, Kenneth Chua, Nigel Ling	Some members lack a theoretical background and tend to delay their tasks. The risk is now reduced because the leads (Kelvin and Kenneth) directly monitor progress, assign clearer tasks, and take over critical work when deadlines are at risk. Nigel assists with coordination.
2.	LinkedIn Learning API credentials not received from client	High	High	High	Kenneth Chua, Kelvin Fong	The LinkedIn Learning integration is dependent on API credentials from the client's IT department. As credentials have not yet been received, this feature cannot be implemented in Sprint 1. The team has followed up with the client and will continue to do so. This task has

						been deferred to a future sprint.
3.	AI API instability or unexpected cost overrun	Medium	High	High	Kenneth Chua	The system depends on Google Gemini API for AI recommendations and learning path generation. If the API becomes unavailable or exceeds the free tier limit, the system may fail. A personal API key is used for development. A fallback algorithm has been implemented for the learning path feature to ensure continuity if the AI service is unavailable.
4.	Insufficient test coverage leading to undetected bugs	Medium	High	Medium	Ivan Hoan, Kelvin Fong	The team has set up Jest and GitHub Actions to automate testing. However, test coverage is currently limited to a few components. The risk is being mitigated by progressively adding unit tests each sprint. Currently 28 tests are passing with zero failures.
5.	Supabase schema changes breaking	Medium	High	Medium	Ivan Hoan, Nigel Ling	The client requested PascalCase for table names and snake_case for column names. Renaming schema references across the

	existing functionality					codebase risks breaking existing queries. This is being mitigated by conducting the migration in multiple passes and verifying all queries after each pass. The migration is currently in progress.
6.	Deployment pipeline instability blocking production releases	Low	High	Medium	Kelvin Fong	The Vercel deployment pipeline encountered two critical blockers this sprint, a lockfile issue and a Next.js security vulnerability. Both were resolved and the website is now live. The risk remains low as long as dependencies are kept up to date and security advisories are monitored regularly.
7.	Website security vulnerabilities exposing sensitive data	Medium	High	Low (now)	Nigel Ling, Kenneth Chua	Previously identified in COS40005 Sprint 3, sensitive credentials were being exposed in browser logs and network requests. This has been resolved this sprint by implementing server side handling for all SIP2 communication and environment variable protection. Further security improvements are

						planned for future sprints.
--	--	--	--	--	--	-----------------------------

3 Sprint progress

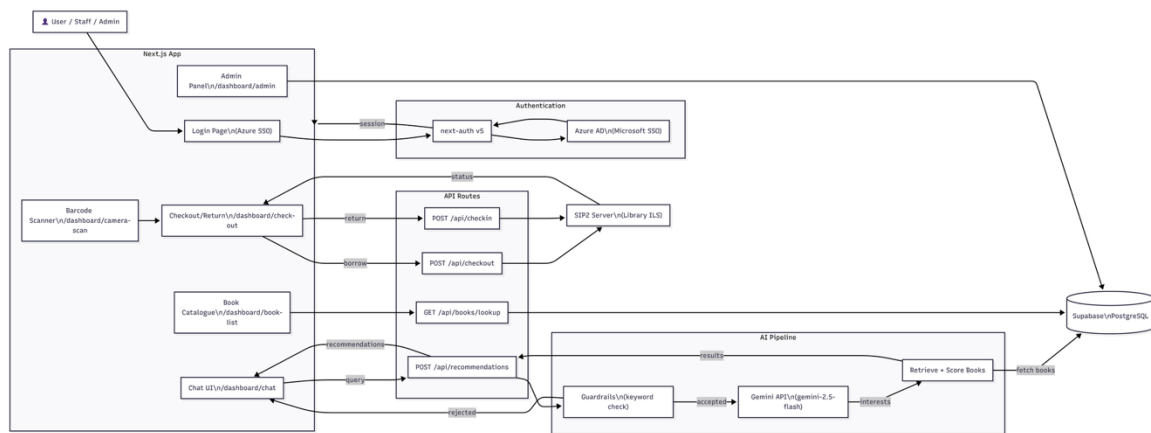


Figure 22: System workflow diagram

This diagram shows the complete system architecture of the Library Self-Checkout System. Users (Student/Staff/Admin) log in through the Login Page, which authenticates via next-auth v5 and Azure AD (Microsoft SSO), then grants access to the app. Once logged in, users interact with four main areas:

1. Barcode Scanner → Checkout/Return page → calls either POST /api/checkin or POST /api/checkout → goes to the SIP2 Server which is the physical library system that processes the actual borrow/return.
2. Book Catalogue → Call GET /api/books/lookup → fetches book data from Supabase.
3. Chat UI → sends a query to POST /api/recommendations → goes through the AI Pipeline (Guardrails check → Gemini extracts interests → Retrieve & Score Books from Supabase) → returns recommendations back to the chat.
4. Admin Panel → reads/writes directly to Supabase.

All data ultimately lives in Supabase (PostgreSQL) — books, users, loans, availability, and tags.

3.1 SPRINT DELIVERABLES

During the period of continuous repair and optimization, an important security hole has been discovered. This hole was revealed by the official React team and impacts all the related frameworks, including the library system that was developed during the project.

In December 2025, a major security flaw was discovered in React. The technical name for this flaw is CVE-2025-55182. The flaw is a remote code execution flaw. In other words, an attacker could execute code on a server remotely from anywhere on the internet without a password or other special access. The flaw is found in versions 19.0 up to 19.2.0 of React that use React Server Components. Due to its severity, it was given a CVSS score of 10 out of 10, which is the highest risk level that can be assigned to a flaw.

The flaw was caused by a problem in how React decodes data that is sent to server function endpoints. An attacker could send a malicious HTTP request. When this request is deserialized by the server, it accidentally executes the attacker's code. The attacker could thus gain full access to the server. The important thing to note is that if a React application doesn't use server functions at all, it is still vulnerable if it allows React Server Components in some way.

This vulnerability was found and announced by security researcher Lachlan Davidson on November 29th. The React team responded quickly to this problem by providing a solution. They published secure versions of React (like 19.0.1, 19.1.2, and 19.2.1), along with information about this vulnerability on December 3rd. All developers who are using frameworks such as Next.js were advised to update their React package as soon as possible.

The correction of this major bug led to the discovery of other related security problems. While reviewing the code of the correction of the bug, new problems were discovered. These problems included how to crash the server (denial of service) and how to leak the source code of an application. As a result, developers had to update their React packages again with versions such as 19.0.4, 19.1.5, and 19.2.4 in order to be totally protected. All of these events show that after a major update of a security problem, one should remain alert because the initial correction of the problem may reveal new problems that need correction.

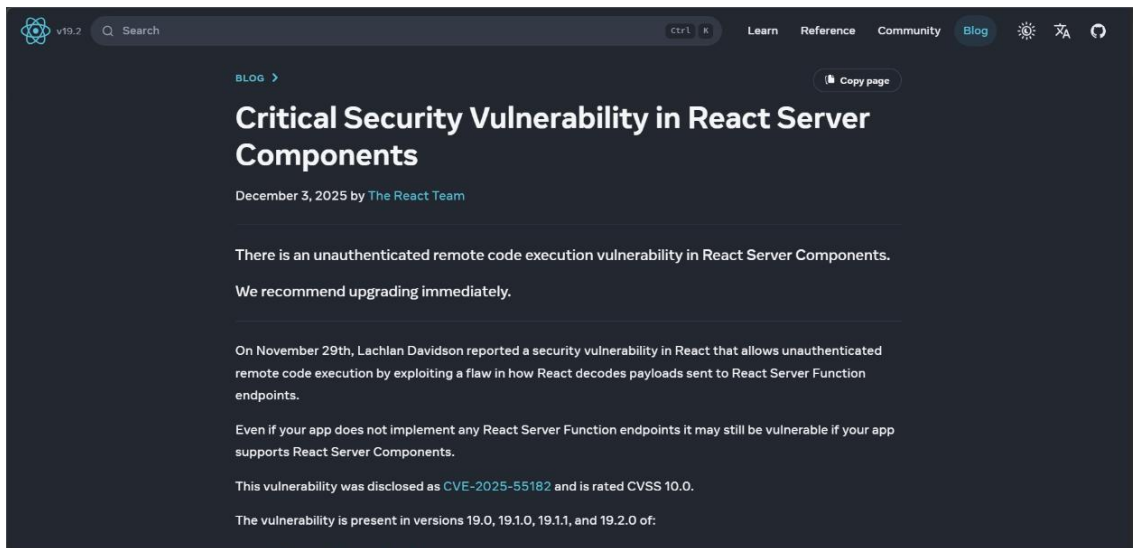


Figure 23: React Official Blog

Critical Security Vulnerability in React Server Components – React

Denial of Service and Source Code Exposure in React Server Components – React

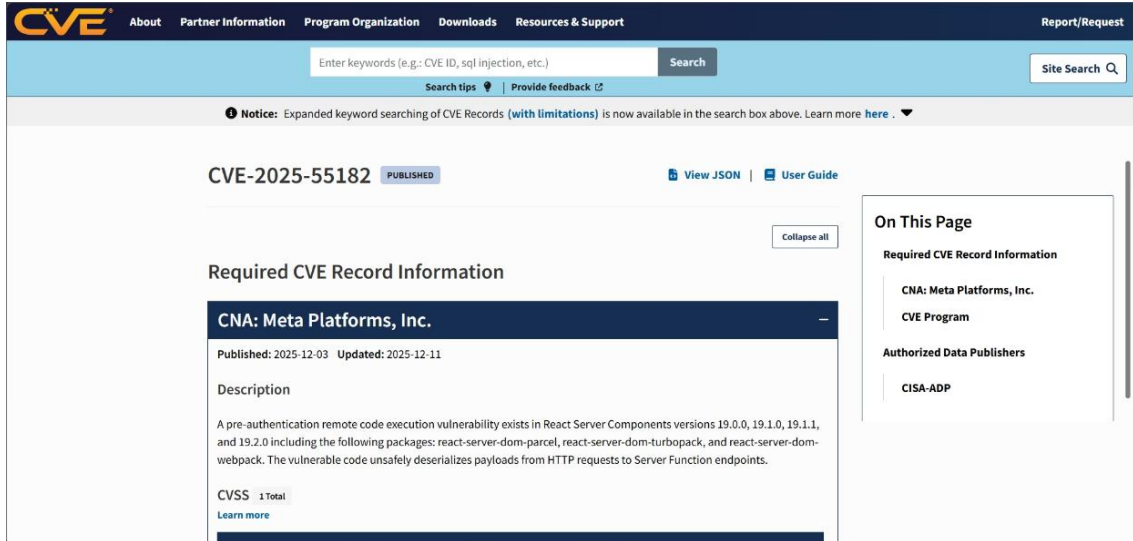


Figure 24: CVE Record Website

CVE Record: CVE-2025-55182

With regard to this vulnerability, various testing methods can be employed, and there are a lot of open-source projects available on GitHub for the same. Simply searching with the corresponding CVE code CVE-2025-55182 will give us various testing methods.

In the experiment, I will use the Python code provided by msanft on GitHub. On his/hers GitHub page, msanft provides an explanation of the principles of the bug and the way the code works.

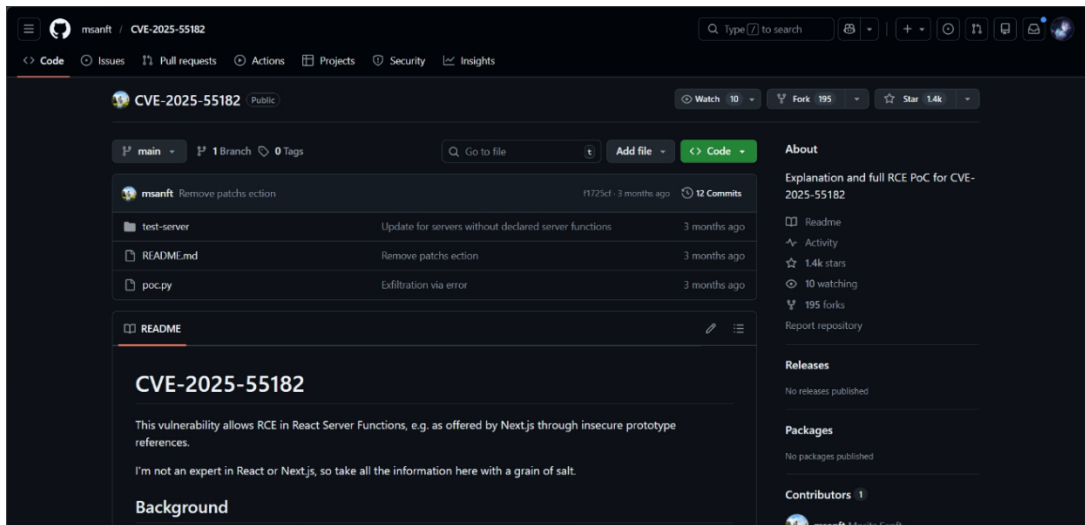


Figure 25: GitHub CVE-2025-55182 testing code

[msanft/CVE-2025-55182: Explanation and full RCE PoC for CVE-2025-55182](#)

The core of this bug lies in a security flaw related to object prototypes – a prototype might be considered a "family blueprint" of a JavaScript object, accessible by all objects by default. Hackers were able to exploit this particular feature.

Step 1: Tampering with an Object's "Default Behavior"

The attacker sends a specially crafted piece of malicious data to the server. The data is designed to exploit a flaw in React's data parsing system. The flaw is that it doesn't verify whether a requested data property is valid. The attacker is able to use a special path like `__proto__` to add a new fake `.then` method to a normal data object. The new method is directly linked to JavaScript's core Function constructor. The Function constructor is a powerful tool that can take a string of code and convert it into a function. It is the raw material that is necessary to perform a remote code execution attack.

Step 2: Sneaking into React's "Async Await" Process

The object that is processed by React's internal mechanisms just so happens to be awaited by the framework, just like Next.js. In JavaScript, if an object is awaited, and that object has a `.then` method, that method will be called automatically. The object that was tampered with by the

attacker in Step 1 was inserted directly into this code path. This causes the fake `.then` method, which is actually referencing the Function constructor, to be called.

Step 3: Using "Second-Level Parsing" to Trigger Malicious Code

The execution of the `.then` method leads the flow of execution into a more complex internal function. This function parses a section of the attacker's data. Within the second-level parsing function, the attacker forges a section of the internal data. For instance, the attacker forges a "form data getter." This forged getter is once again executed against the Function constructor. The attacker sends their malicious system command (for instance, a simple instruction to open a calculator app) as a "data key name."

Lastly: Command Execution

As a result of the second-level parsing operation, as it attempts to retrieve data from the fake "form data getter," it instead invokes the Function constructor to create a new function. The "code template" it uses to accomplish this is none other than the system command that was provided by the attacker. The new function is then awaited, thus causing it to execute on the server as desired by the attacker.

In simple terms, the attacker took advantage of a small mistake in React's data processing assembly line. By changing an object's "genetic blueprint," he managed to add it to the assembly line. After that, he used some of the existing "robotic arms" (functions) on the assembly line to package his own instructions in such a way that they looked like a legitimate component. This component was then assembled and executed within the server.

```
(base) PS C:\WINDOWS\system32> cd 'C:/Users/User/Downloads'
(base) PS C:\Users\User\Downloads> python poc.py http://localhost:3000/ "hostname"
500
0:{"a":"$01", "f": "", "b": "development"}
1:E{"digest": "MSiKatanaUmbra", "name": "Error", "message": "NEXT_REDIRECT", "stack": [], "env": "Server"}

(base) PS C:\Users\User\Downloads> python poc.py http://localhost:3000/ "mkdir new_file"
500
0:{"a":"$01", "f": "", "b": "development"}
1:E{"digest": "3006528253", "name": "Error", "message": "NEXT_REDIRECT", "stack": [], "env": "Server"}

(base) PS C:\Users\User\Downloads> python poc.py http://localhost:3000/ "mkdir newFile"
500
0:{"a":"$01", "f": "", "b": "development"}
1:E{"digest": "1534190514", "name": "Error", "message": "NEXT_REDIRECT", "stack": [], "env": "Server"}

(base) PS C:\Users\User\Downloads> |
```

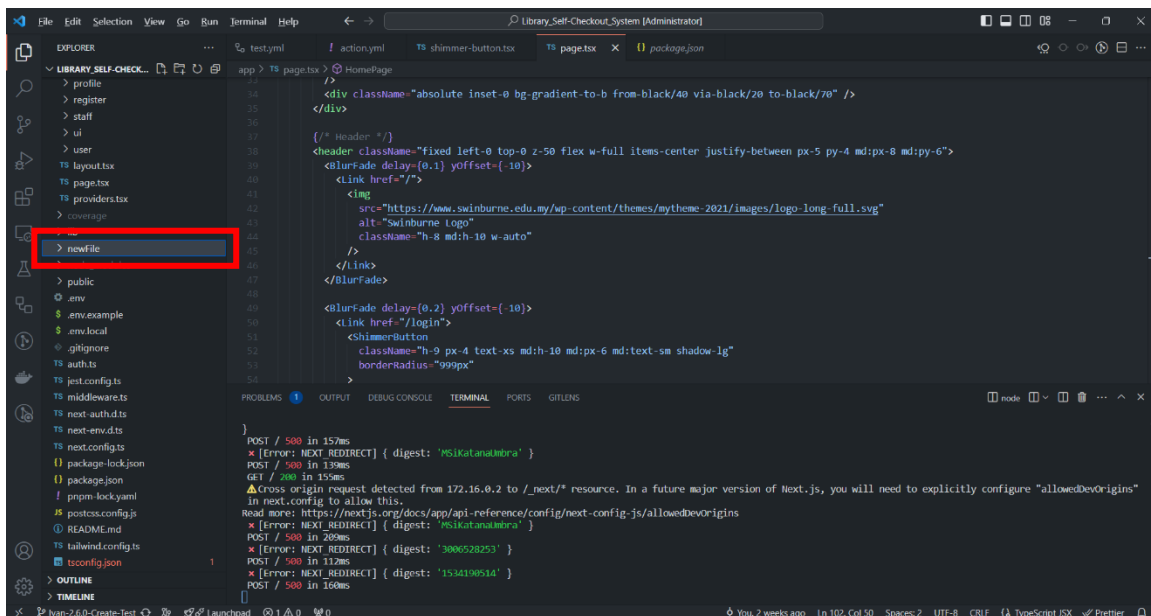


Figure 26: Add a new file with command prompt

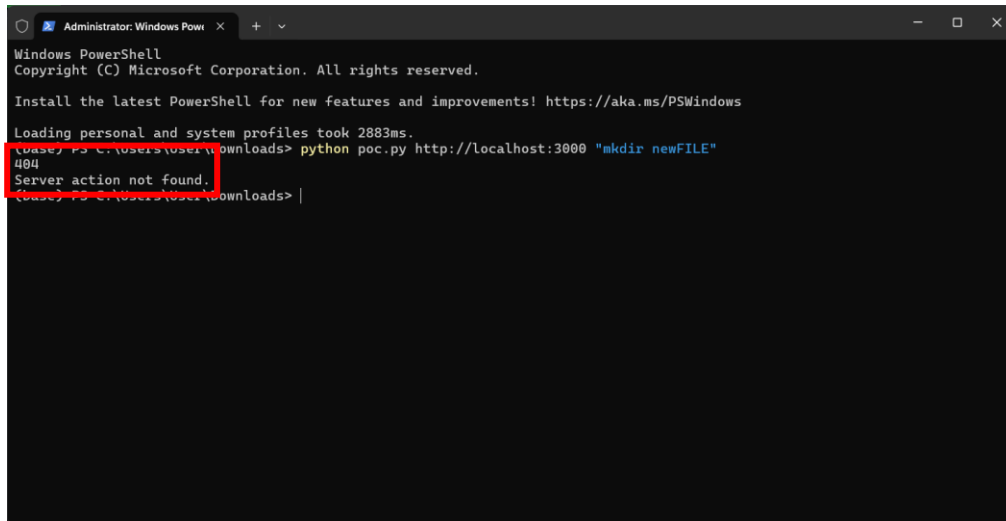
In this vulnerability test, I launched our website in Visual Studio Code and kept it running.

Next, I opened the built-in PowerShell on computer to carry out the vulnerability exploitation. Following the instructions on GitHub, I ran the downloaded Python script.

We can use the Python script directly to manipulate our source files via cmd commands. In PowerShell, within our project folder, I created a new folder named 'newFile'.

As shown in the screenshot, we can see that our source files have been added to this folder; no login or permissions are required to add files directly.

Testing has confirmed that this vulnerability can be exploited across different devices. I can run the website on Device A whilst exploiting the vulnerability on Device B, again without requiring any permissions. If we deploy our website whilst this vulnerability remains unpatched, malicious actors could exploit it to directly modify files.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

Loading personal and system profiles took 2883ms.
C:\Users\User\Downloads> python poc.py http://localhost:3000 "mkdir newFILE"
404
Server action not found.
C:\Users\User\Downloads> |
```

Figure 27: Failed to add the folder

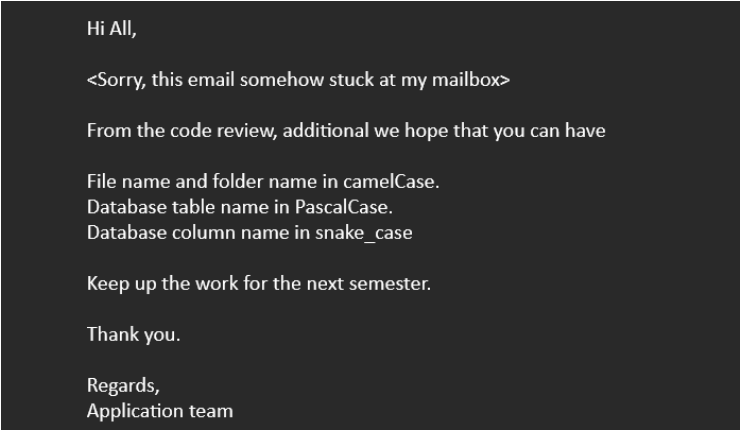
Once this vulnerability is patched, if we try to exploit it in the same way, we get a 404 Error. This means that we can no longer directly edit the source code as we used to.

In accordance with the client's requirements, and in order to standardise the naming convention, the client has requested that we adopt a consistent naming scheme.

With respect to file names, we have been asked to use camelCase. To achieve this, we need to join all the words together and make the first letter of each word lowercase and the remainder uppercase.

In accordance with this requirement, we have renamed all files and folders to use the camelCase naming convention, with the exception of certain files that cannot be renamed (such as JSON and YML files).

As per the client's requirements, we shall also change the format of our Supabase tables. The format for the tables' names is PascalCase, which means we capitalize the first letter of all words. On the other hand, column names are written in snake_case, which means we join words together using underscores. However, as the data in our Supabase database tables is currently vast and involves a great deal of content, we have not yet completed all the formatting adjustments.

A screenshot of an email from a client, displayed on a dark background. The text is white and includes a greeting, an apology for a delayed response, a list of naming conventions for files, folders, database tables, and columns, a closing statement about future work, and a sign-off.

Hi All,

<Sorry, this email somehow stuck at my mailbox>

From the code review, additional we hope that you can have

- File name and folder name in camelCase.
- Database table name in PascalCase.
- Database column name in snake_case

Keep up the work for the next semester.

Thank you.

Regards,
Application team

Figure 28: Client Email

This branch is 61 commits ahead of and 2 commits behind `main`.

Contribute

Name	Last commit message	Last commit date
..		
communities	Change the file to camelCase	2 weeks ago
learning	Change the file to camelCase	2 weeks ago
recommendations	Change the file to camelCase	2 weeks ago
activeLoansTable.tsx	Change the file to camelCase	2 weeks ago
bookCatalogTable.tsx	Change the file to camelCase	2 weeks ago
bookItemsFilter.tsx	Change the file to camelCase	2 weeks ago
bookList.tsx	Change the file to camelCase	2 weeks ago
bookListMobile.tsx	Change the file to camelCase	2 weeks ago
cameraScanner.tsx	Change the file to camelCase	2 weeks ago
cameraScannerButton.tsx	Change the file to camelCase	2 weeks ago
checkInForm.tsx	Change the file to camelCase	2 weeks ago
checkOutForm.tsx	Change the file to camelCase	2 weeks ago
createBookForm.tsx	Change the file to camelCase	2 weeks ago
dashboardShell.tsx	Change the file to camelCase	2 weeks ago

Figure 29: camelCase name

FYP FREE FYP TEST main PRODUCTION Connect Feedback Search...

Table Editor

schema public

New table

Search tables...

BookTags (UNRESTRICTED)

BookTagsLi... (UNRESTRICTED)

Books (UNRESTRICTED)

Copies (UNRESTRICTED)

Loans (UNRESTRICTED)

Users (UNRESTRICTED)

active_loans (UNRESTRICTED)

audit_log (UNRESTRICTED)

available_copies (UNRESTRICTED)

book_recommendation (UNRESTRICTED)

book_tag_links (UNRESTRICTED)

book_tags (UNRESTRICTED)

books (UNRESTRICTED)

communities (UNRESTRICTED)

community_book_re... (UNRESTRICTED)

community_ma... (UNRESTRICTED)

community_post_c... (UNRESTRICTED)

Filter by book_id, tag_id or all

Sort

RLS disabled

Index Advisor

Enable Realtime

Postgres

Insert

book_id uuid

tag_id uuid

This table is empty

Import data from CSV

or drag and drop a CSV file here

Page 1 of 1 100 rows 0 records

Data Definition

Figure 30: Supabase Table

Within the Next.js framework, several testing tools are officially recommended: Jest, Vitest, Cypress, and Playwright.

For this project, we have opted to use Jest as our website testing tool. Additionally, during this sprint, we have created a simple GitHub Action to run our Jest tests.

The reason behind the usage of Jest in Next.js is that Jest can be utilized in order to ensure that there is automation testing in your application. Jest is an extensive testing framework that can be combined with React Testing Library in order to ensure that user interactions with your React-based application are working in an expected manner. Because of the fact that there are significant optimizations provided in Next.js in terms of Jest, developers can easily write tests in order to ensure that there are no logical issues in the code, and refactoring the code or adding new features does not impact existing features in an undesired manner.

GitHub Actions is a powerful automation platform that is integrated directly into GitHub repositories and is intended for use by developers for automation of their workflow processes. The main purpose of using GitHub Actions is to help users automate their Continuous Integration and Continuous Delivery processes directly from their GitHub repositories. This means that users can, for example, automate running tests for each and every new pull request that is made, as well as deploying a given application directly to a production server after it is merged into the codebase. Apart from the automation of CI/CD processes, the utility of GitHub Actions can be used for automating project management processes, such as the creation of labels, handling of dependency updates, and creation of release notes.

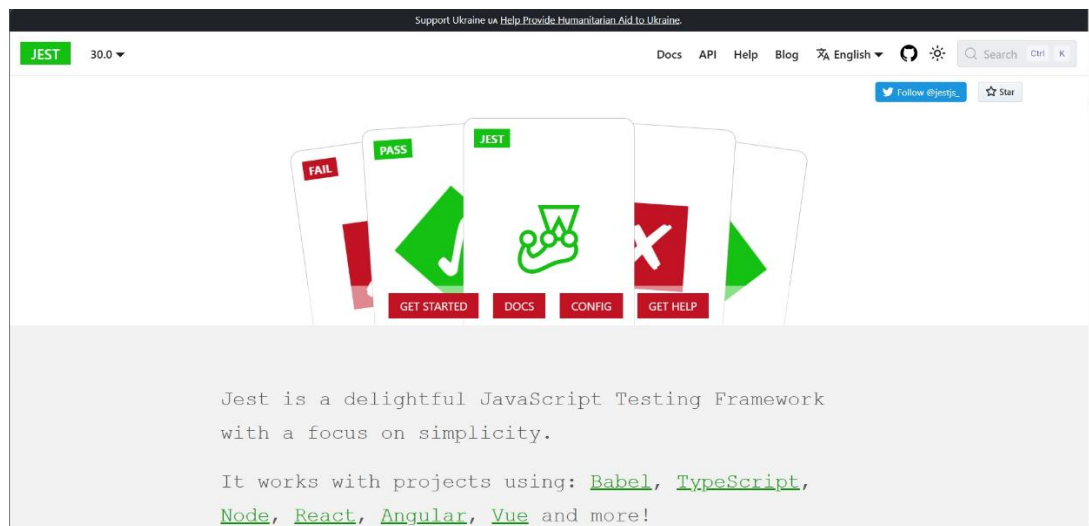


Figure 31: Jest Test

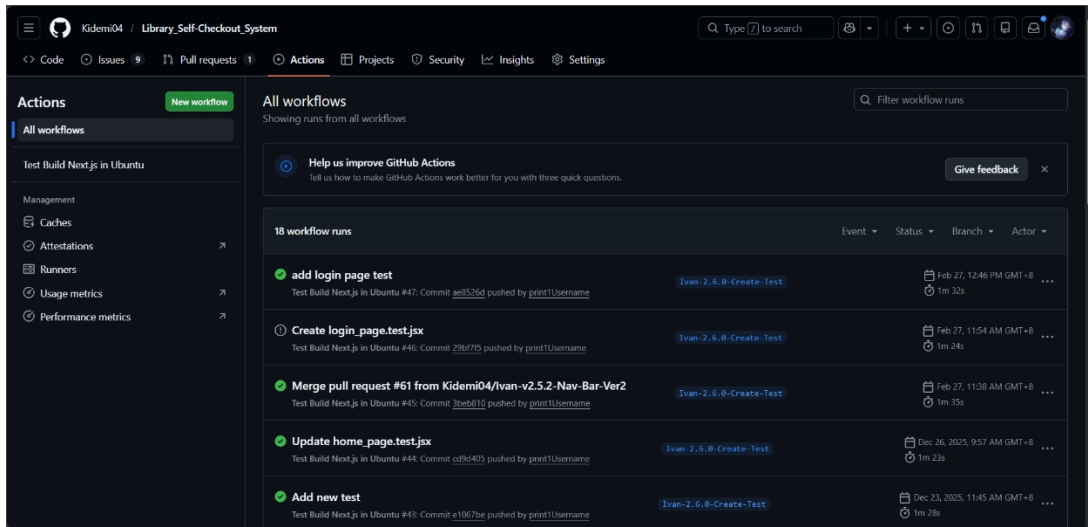


Figure 32: GitHub Action

In previous versions, there was a chance that a bug causing the dark theme to appear would occur upon first login; this bug has now been fixed following changes to the theme logic.

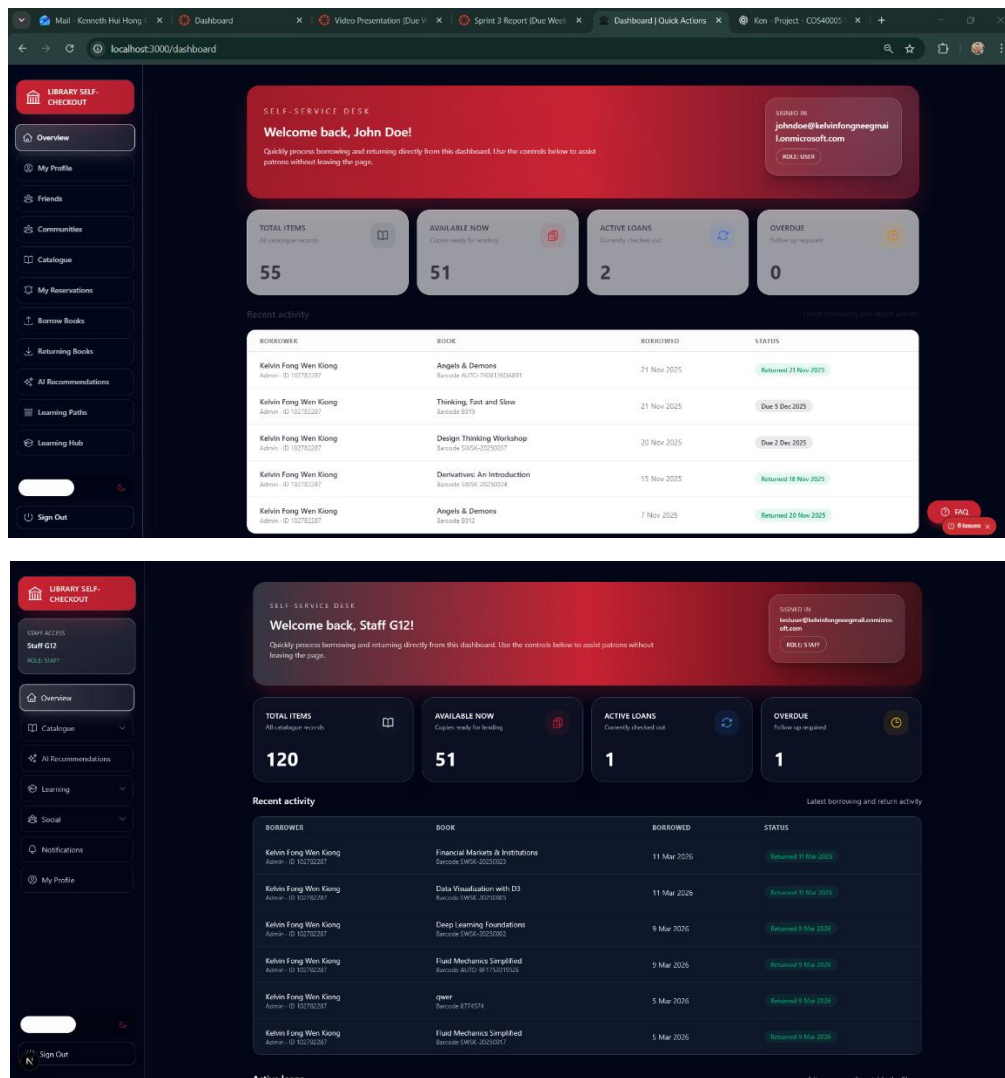


Figure 33: Dark Theme bug fixed and optimize

In order to enable the mobile user to easily adjust and get accustomed to our system, and in order to make our system user-friendly, we have optimized the display in mobile mode. The interface has been designed in a way that is most common in most mainstream software programs.

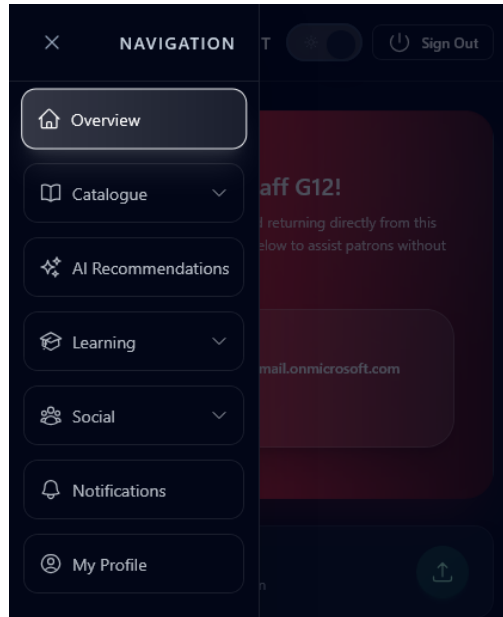


Figure 34: New version Navigation bar

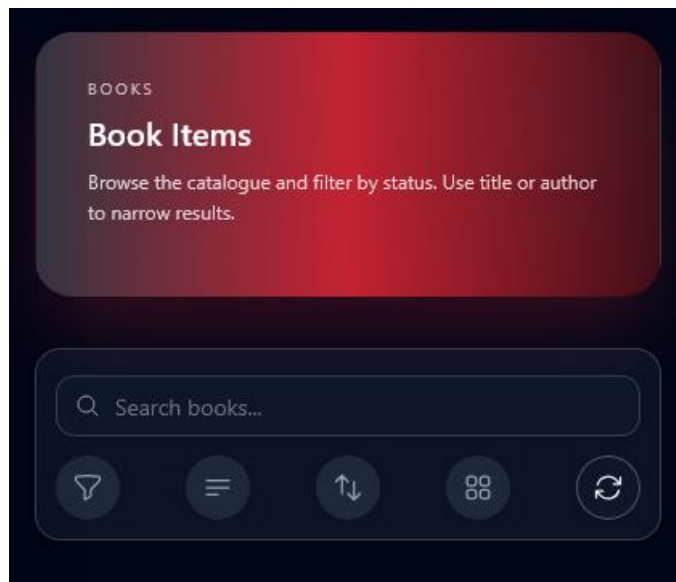


Figure 35: Mobile version search bar

Api Emulator

Successfully perform CheckIn & CheckOut without showing sensitive data. Sensitive data such SIP key, patron password only shown in API emulator

TIMESTAMP	ROUTE	STATUS	METHOD	REQUEST BODY	RESPONSE
3/9/2026, 6:05:21 PM	checkin	200 - SUCCESS	POST	View Request	Check-in processed successfully
3/9/2026, 5:58:13 PM	checkin	500 - ERROR	POST	View Request	transactionDate must be in format YYYYMMDDZZZZHHMMSS, returnDate must be in format YYYYMMDDZZZZHHMMSS
3/9/2026, 5:57:20 PM	checkin	500 - ERROR	POST	View Request	transactionDate must be in format YYYYMMDDZZZZHHMMSS, returnDate must be in format YYYYMMDDZZZZHHMMSS
3/9/2026, 5:31:46 PM	checkin	500 - ERROR	POST	View Request	transactionDate must be in format YYYYMMDDZZZZHHMMSS, returnDate must be in format YYYYMMDDZZZZHHMMSS
3/9/2026, 5:31:22 PM	checkout	500 - ERROR	POST	View Request	transactionDate must be in format YYYYMMDDZZZZHHMMSS, returnDate must be in format YYYYMMDDZZZZHHMMSS, noDueDate must be in format YYYYMMDDZZZZHHMMSS

TIMESTAMP	ROUTE	STATUS	METHOD	REQUEST BODY	RESPONSE
3/19/2026, 8:33:07 PM	checkout	200 - SUCCESS	POST	View Request <pre>{ "scRenewalPolicy": "Y", "noBlock": "N", "transactionDate": "2026031908002033", "returnDate": "202604020800000000", "institutionId": "LIB001", "patronIdentifier": "102782287", "itemIdentifier": "CUCV-20250004"}</pre>	Checkout processed successfully

TIMESTAMP	ROUTE	STATUS	METHOD	REQUEST BODY	RESPONSE
3/9/2026, 6:16:37 PM	checkin	200 - SUCCESS	POST	View Request <pre>{ "noBlock": "N", "transactionDate": "202603090800181", "returnDate": "202603090800181637", "currentLocation": "Main Library", "institutionId": "LIB001", "itemIdentifier": "SWSK-20250002", "patronIdentifier": "102782287"}</pre>	Check-in processed successfully

Figure 36: API Logs Viewer

Learning Path System

For the current stage, users can input a topic or select from preset chips (Data Science, AI, Cybersecurity, Software Engineering, Business Analytics, Robotics) and the system generates a structured 3-stage learning path (Beginner → Intermediate → Advanced) using books from the library catalogue. The stage classification is powered by the Gemini AI model (gemini-2.5-flash), which intelligently assigns each book to the appropriate difficulty level and generates a short reason explaining why it suits that stage. A fallback mechanism is also in place whereby if Gemini is unavailable, the system automatically distributes books evenly across the three stages to ensure the feature remains functional at all times. Each book card displays real-time availability status fetched directly from the Supabase database, indicating either the number of available copies or an "On loan" status. A dedicated API endpoint POST /api/learning-path was developed to handle the full pipeline from receiving the topic input, retrieving relevant books from the catalog, classifying them into stages, and returning the structured learning path to the frontend. The previously static learning paths page was replaced with this fully dynamic AI-powered generator.

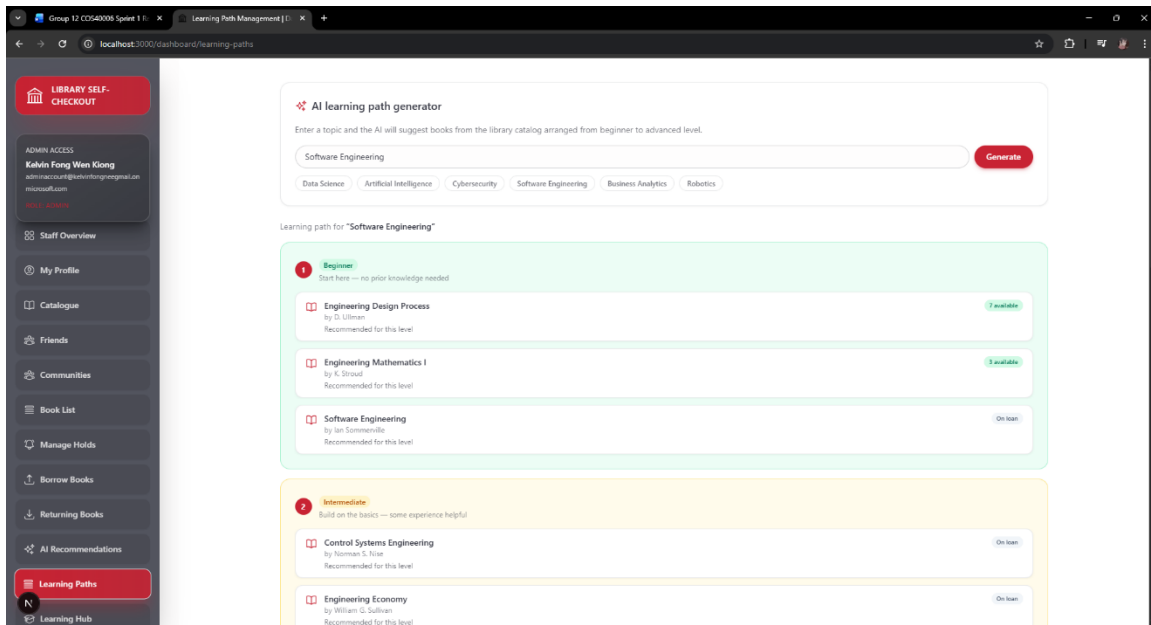


Figure 37: Learning Path System

AI-powered book recommendations

The AI-powered book recommendation feature is a conversational interface embedded within the student dashboard that allows authenticated users to receive personalised book suggestions by describing their interests or study needs in natural language. The system operates as a multi-stage pipeline integrating a large language model, a relational book catalogue, and a set of safety guardrails, all coordinated through a single API endpoint at `/api/recommendations`. catalogue, and a set of safety guardrails, all coordinated through a single API endpoint at `/api/recommendations`.

User Interface

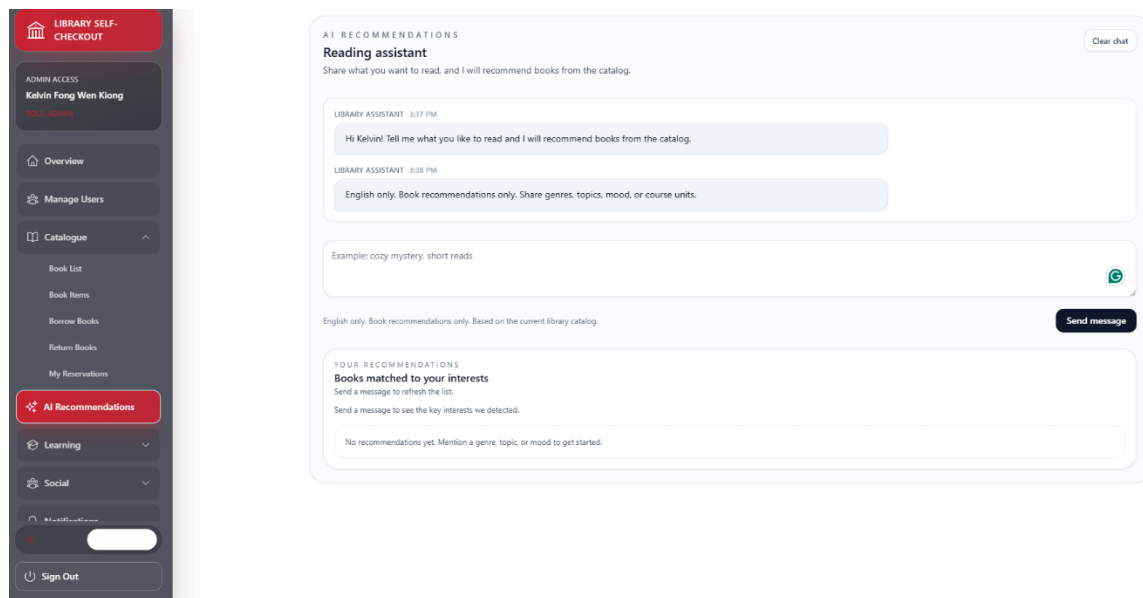


Figure 38: AI Recommend User Interface

The student-facing interface is implemented as a React client component rendered within the `/dashboard/recommendations` route. Upon loading, the page presents a chat pane personalised with the student's display name alongside a two-column book grid that populates dynamically as recommendations are generated. Each book card displays the title, author, availability status, and a one-sentence explanation of why the book was recommended, giving the student immediate context for each suggestion. Below the chat input, active interest tags summarise the keywords extracted from the student's most recent query, providing visual feedback on how the system interpreted their request. A typing indicator appears while the assistant is processing, and the interface auto-scrolls to the latest message. Students may clear the conversation at any time using a dedicated button, and a rate limit notice is surfaced inline when the request threshold is reached. Input submission is handled via the Enter key, with Shift+Enter reserved for inserting line breaks.

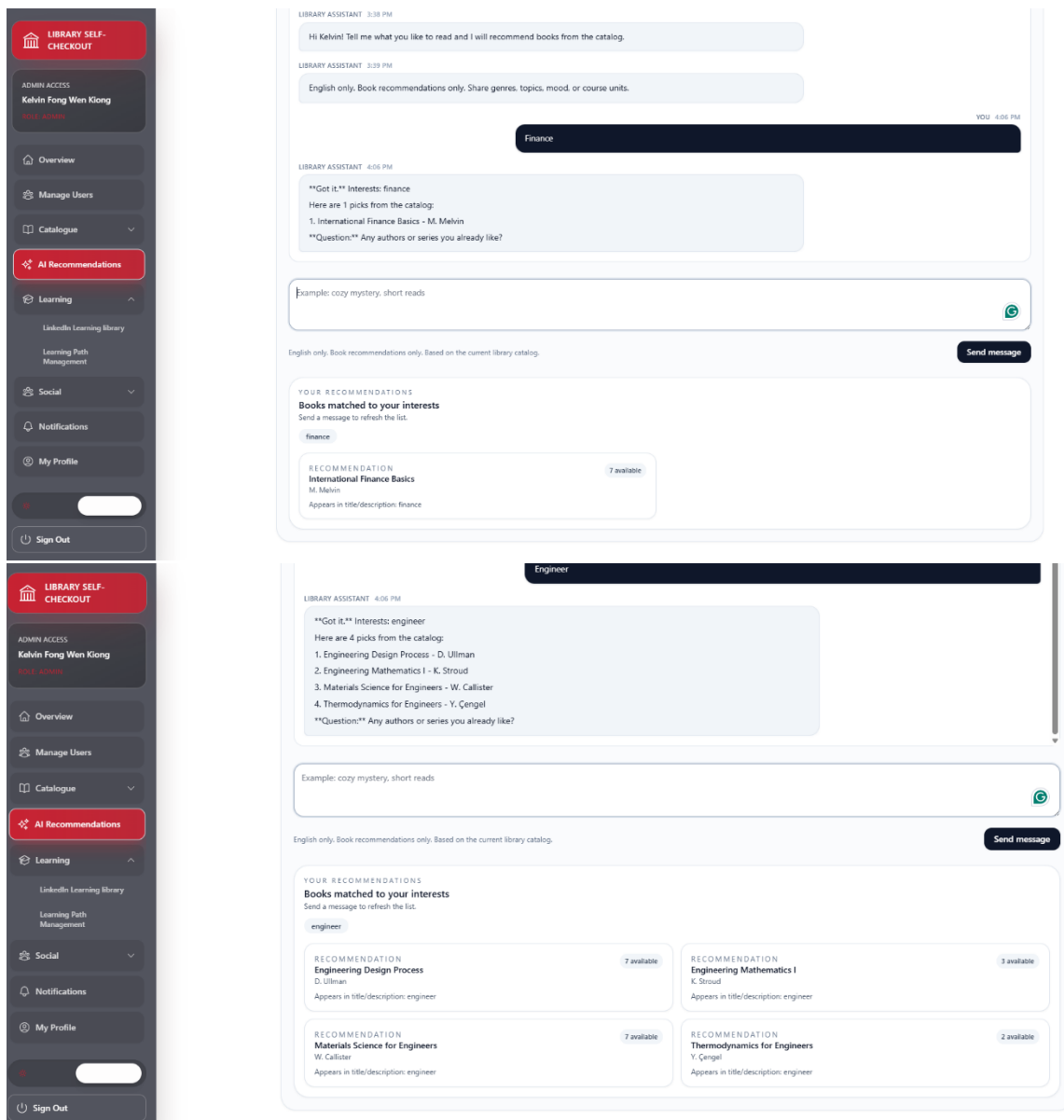


Figure 39: Ai Recommend Chat Box

AI Model Integration and Architecture

The system integrates Google Gemini 2.5 Flash as its primary language model, configured with a temperature of 0.2 to produce focused and deterministic outputs, and a maximum output token limit of 220 per response. The overall architecture follows a Retrieval-Augmented Generation (RAG) pattern: rather than relying on the model's parametric knowledge to identify books, the system uses the model exclusively for natural language understanding tasks, specifically, extracting structured user preferences from free-text input, while all book retrieval is performed against a live Supabase PostgreSQL catalogue. This design ensures that recommendations reflect the actual holdings of the library rather than titles the model may have encountered during training. In the event that the Gemini API is unavailable or returns a 403 error, the system falls back to a deterministic tokenisation routine and a set of eleven canned chat responses, allowing the feature to remain partially functional without the model.

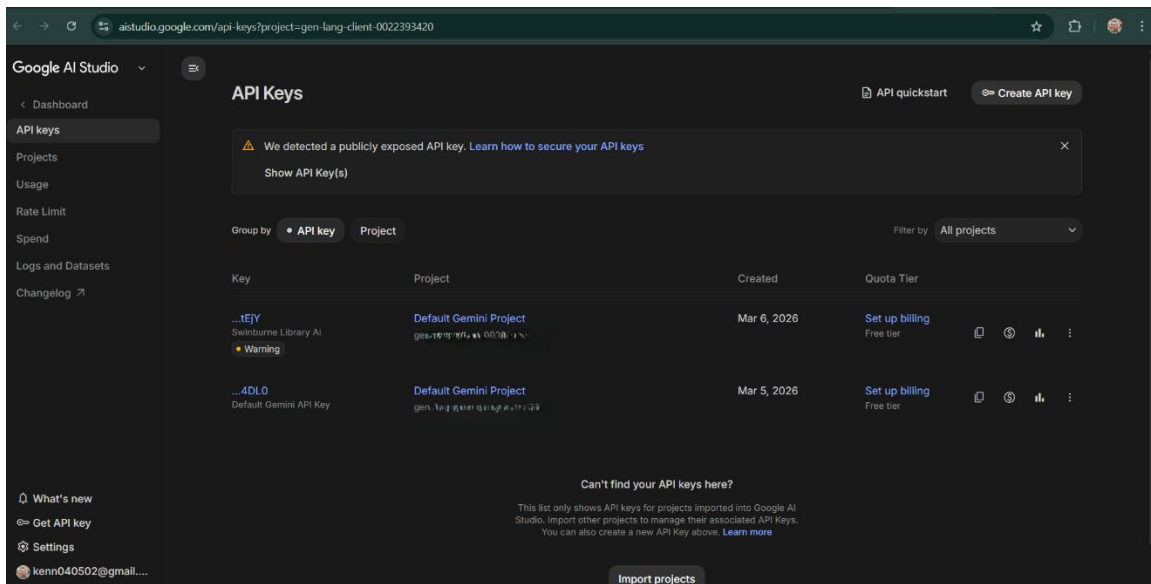


Figure 40: API Keys

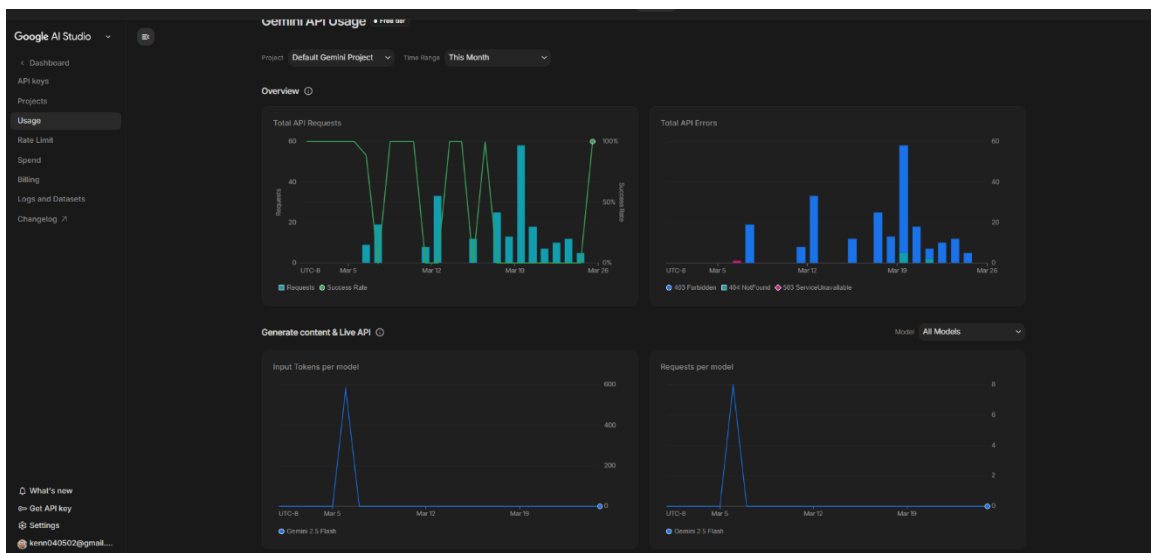


Figure 41: API Keys Usage

Recommendation Pipeline

The recommendation pipeline consists of nine sequential stages executed on each valid request. First, the system enforces a sliding-window rate limit on a per-client IP basis, permitting a maximum of five requests per ten-second window with a minimum inter-request interval of 1,200 milliseconds. Requests that arrive too frequently receive a `rate_limited` response kind without entering the remainder of the pipeline.

For requests that pass rate limiting, the system determines whether the input is a conversational query or a book recommendation request. Chat-intent messages, such as study questions related to computing, mathematics, or language, are routed to a separate `answerQuestion` function backed by the same Gemini model, which returns a concise answer constrained to those subject areas. All other inputs proceed to the guardrail evaluation stage, where a multi-layer content check classifies the request as `accept`, `clarify`, or `reject`. This stage detects non-ASCII characters and common foreign greeting patterns to restrict input to English, identifies greetings and small talk that should be acknowledged rather than interpreted as book queries, and filters a set of sensitive topics including political content, religious discussion, NSFW material, credential requests, and attempts to probe the model's identity. Inputs flagged as sensitive are rejected with an explanatory message; ambiguous or unclear inputs receive a clarifying follow-up question.

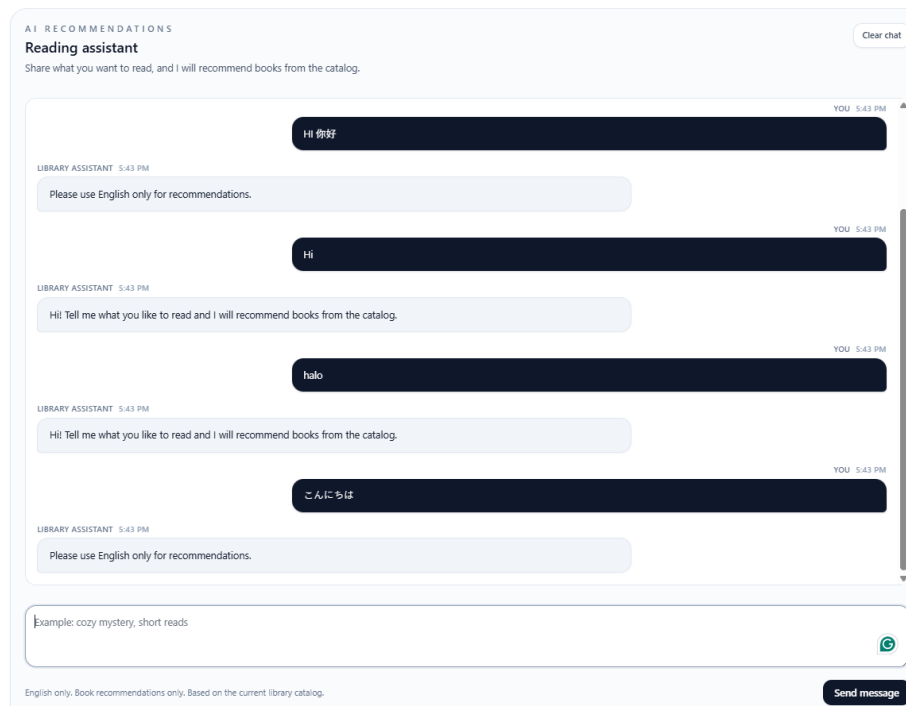


Figure 42: AI Recommend Chat with other language

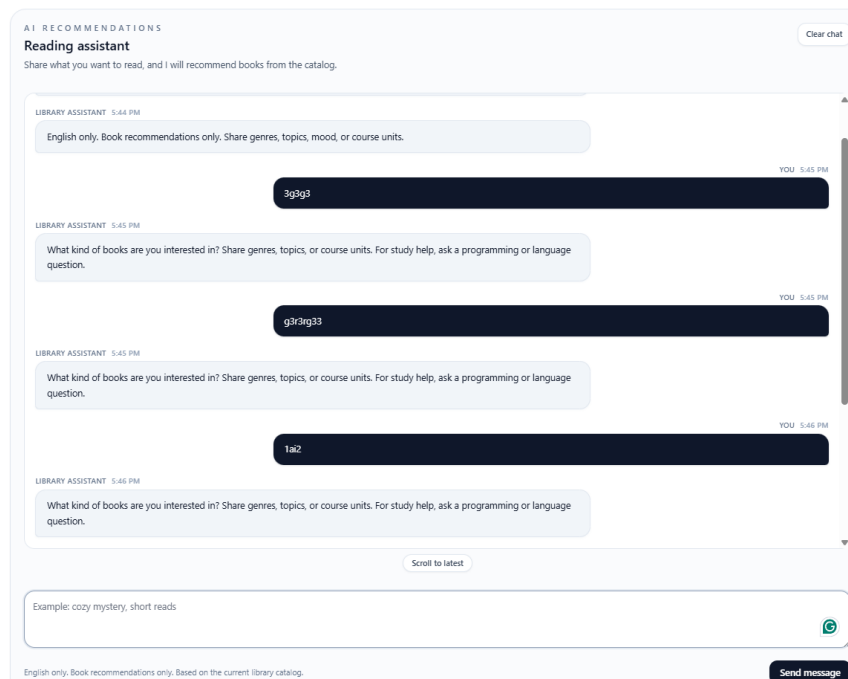


Figure 43: AI Recommend Chat Disable invalid input

Inputs that pass guardrail evaluation proceed to preference extraction, where Gemini is prompted with a structured JSON output schema requiring a plain-language summary of the user's interest, an array of three to eight normalised interest tokens, and an optional follow-up question. These tokens are then expanded using a synonym map, for example, the token `ai` expands to machine, learning, data, and intelligence, before being used to query the Supabase catalogue. The retrieval stage issues a primary keyword query against the database and falls back to the full catalogue if fewer than forty candidate books are returned. Each candidate is then scored across multiple fields: title matches contribute six points, tag matches five, author matches four, classification three, publisher two, and ISBN or corpus matches one to two points. The top two hundred scored candidates are passed to the ranking stage.

The ranking stage applies a composite scoring function that awards three points for direct tag matches against the extracted interest tokens, two points for tags matched via association rules, 1.5 points for text corpus matches, one point for catalogue availability, up to 2.5 points for popularity, and up to 0.5 points for recency. Association rules are mined at runtime from tag co-occurrence data across the catalogue, retaining rules with a support of at least 0.02 or a confidence of at least 0.15, capped at the top four rules per source tag. Finally, a diversification pass limits the result set to a maximum of two books per author and deduplicates by title before returning the ranked list to the client.

Limitations

Despite the breadth of the pipeline, several limitations remain in the current implementation. The preference extraction prompt and token-scoring logic do not cleanly handle multi-topic queries;

when a student expresses interest in two unrelated subjects simultaneously, the extracted tokens tend to blend both topics rather than treating them as distinct retrieval criteria, which can dilute the relevance of results. The system also has no mechanism for honouring negative constraints, a request such as "something about data science but not Engineer" will ignore the exclusion and may still surface Engineer focused titles. Because conversation history is not persisted between sessions, a student who returns after closing the browser begins with no memory of prior exchanges, limiting the system's ability to learn preferences over time. The rate limiter is implemented entirely in memory, meaning that its state is not shared across server instances or preserved across deployments; in a horizontally scaled or restarted environment, rate limit counters reset silently. Finally, several tokens such as " Engineer " and " Engineering" require an explicit computing-context hint to disambiguate correctly, and without that hint the retrieval stage may produce off-target results for queries that use those terms colloquially.

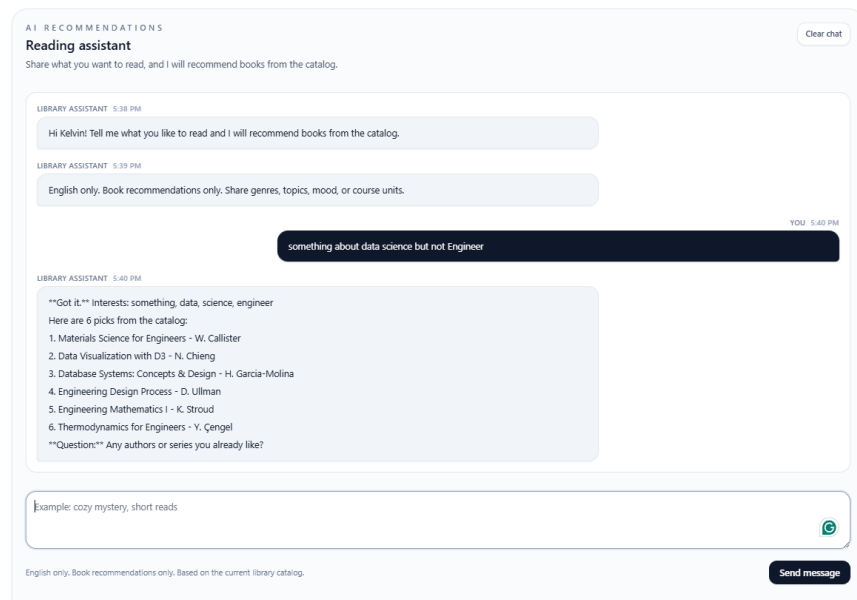


Figure 44: Negative response message of AI limitation

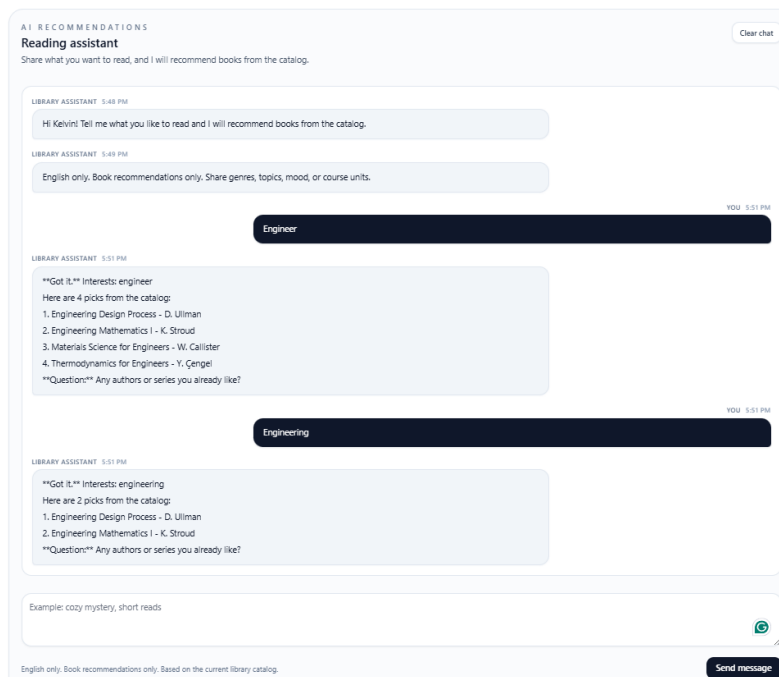


Figure 45: Different typo of AI limitation

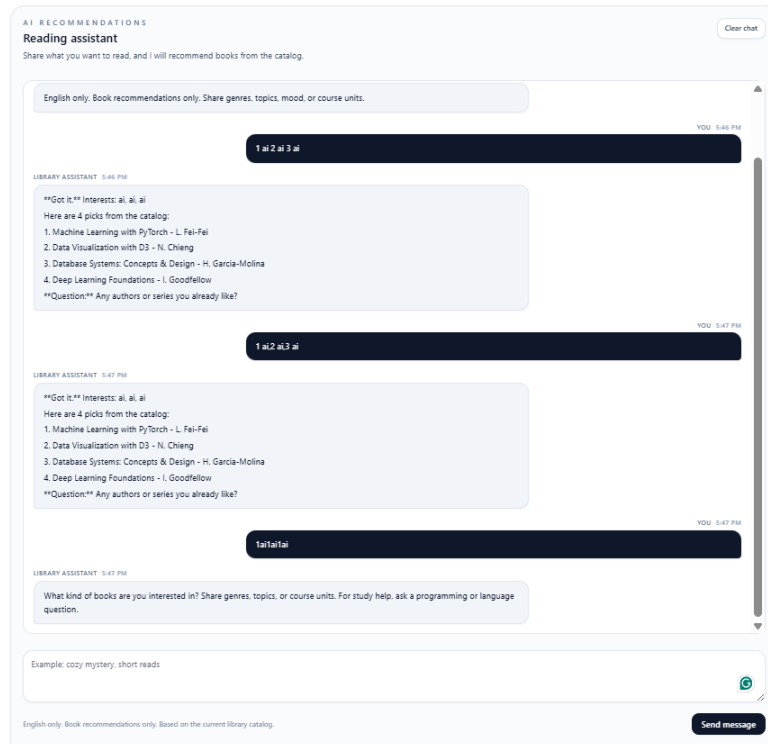


Figure 46: Space Gap typo of AI limitation Input

LinkedIn Learning integration

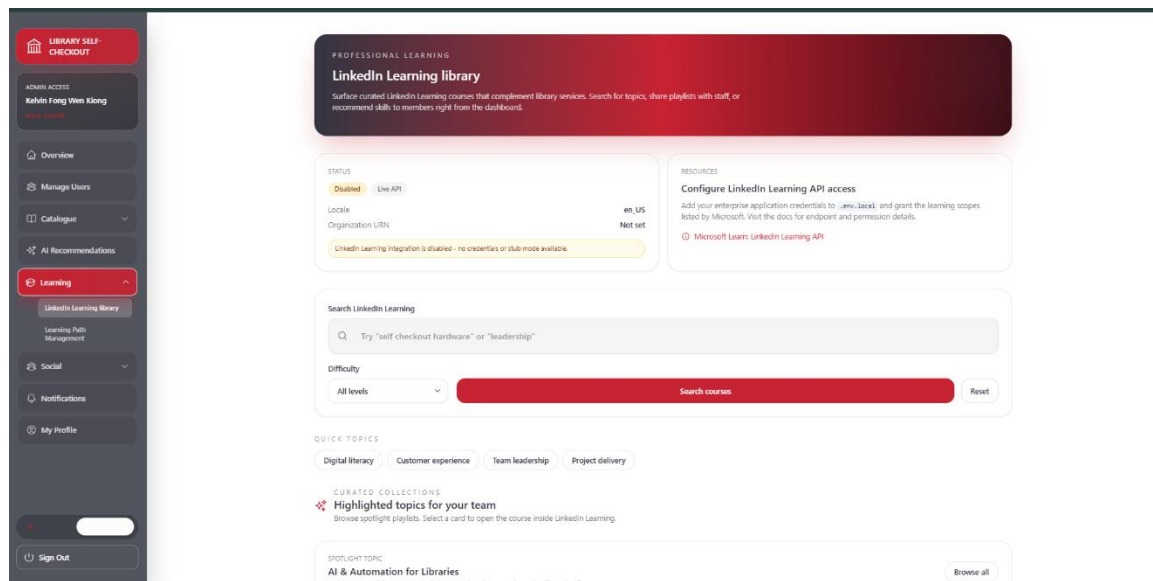


Figure 47: UI of LinkedIn Learning library

The LinkedIn Learning integration is a professional development feature embedded within the student and staff dashboard at `/dashboard/learning/linkedin`. It surfaces curated and searchable LinkedIn Learning courses directly within the library portal, enabling users to discover structured online learning content alongside the library's physical and digital collection. The page operates in two modes depending on whether a search query is present. Without a query, it renders three hardcoded curated topic collections, AI & Automation for Libraries, Experience Design & Service Quality, and Leadership & Upskilling, each populated with up to four course cards, alongside quick-filter buttons for commonly relevant topics such as digital literacy, customer experience, team leadership, and project delivery. When a user submits a search, the page transitions to display up to twelve results filtered by the entered keywords and an optional difficulty level chosen from Beginner, Intermediate, and Advanced. Each course is shown as a responsive card containing the course thumbnail or a placeholder, a content-type badge, the title, a truncated description, formatted duration, difficulty badge, up to three topic tags, the author name, and a link that opens the course on the LinkedIn Learning platform. Staff and administrator accounts additionally see a status panel at the top of the page showing whether the integration is live or running on sample data, the configured locale, and the organisation URN, along with a link to the Microsoft Learn configuration documentation.

The feature is backed by a server-side service module that authenticates against the LinkedIn Learning API using the OAuth 2.0 client credentials flow, caching the access token in memory with automatic refresh sixty seconds before expiration. Course searches are issued against LinkedIn's `/rest/learningAssets` endpoint, and the raw responses are normalised through a set of transformation helpers that handle variations in field naming across API response shapes, producing a consistent internal type covering title, description, duration, difficulty,

topics, authors, thumbnail, and content type. When API credentials are absent or stub mode is explicitly enabled, the system falls back automatically to six bundled sample courses tailored to library contexts, applying the same tokenised search and difficulty filtering logic so the interface renders identically regardless of the data source.

```
426 // total: number,
427 };
428 };
429
430 const fetchLinkedInLearningAssets = async (
431   params: URLSearchParams,
432 ): Promise<RawLearningAssetResponse> => {
433   const token = await getAccessToken();
434   const url = new URL('rest/learningAssets', `${env.apiBase}/`);
435   url.search = params.toString();
436
437   const response = await fetch(url.toString(), {
438     method: 'GET',
439     headers: {
440       Authorization: `Bearer ${token}`,
441       'LinkedIn-Version': env.apiVersion,
442       'X-Restli-Protocol-Version': '2.0.0',
443       Accept: 'application/json',
444     },
445     cache: 'no-store',
446   });
```

Figure 48: learningAssets part

The primary limitation of the current implementation is that the live API integration has not been validated end-to-end in the production environment, as the required enterprise LinkedIn Learning credentials have not yet been provisioned for the project. All functionality visible during the sprint therefore runs against the bundled sample data. The six-item sample dataset also constrains the usefulness of search and filtering, as many queries return no results where a full catalogue would not. Beyond the data access limitations, the token cache is held in process memory and does not survive server restarts or scale across multiple instances, and the curated topic collections and quick-filter labels are hardcoded in the page component, requiring a code change to update them rather than a configuration edit.

3.2 REPOSITORY AND WORKFLOW EVIDENCE

3.2.1 GitHub

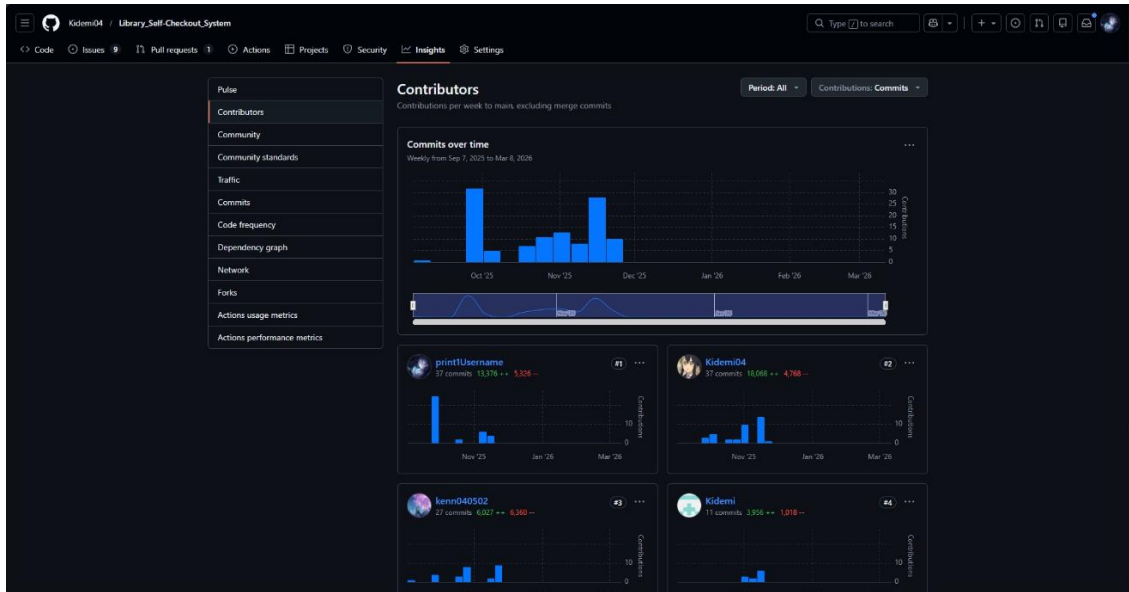


Figure 49: GitHub Contribution

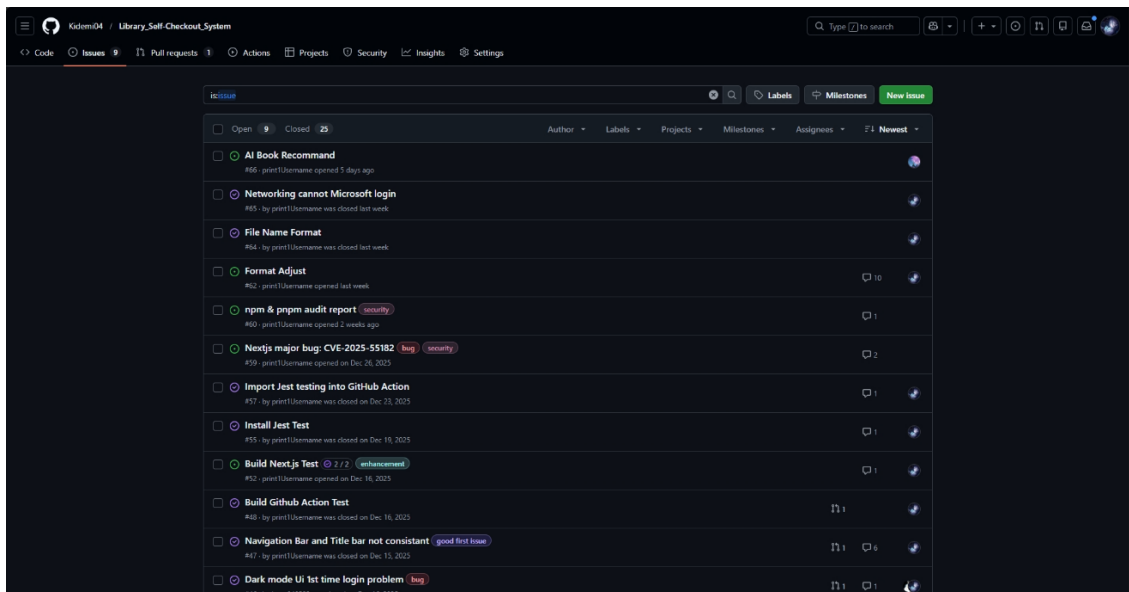


Figure 50: GitHub Issue

3.2.2 WhatsApp Chat

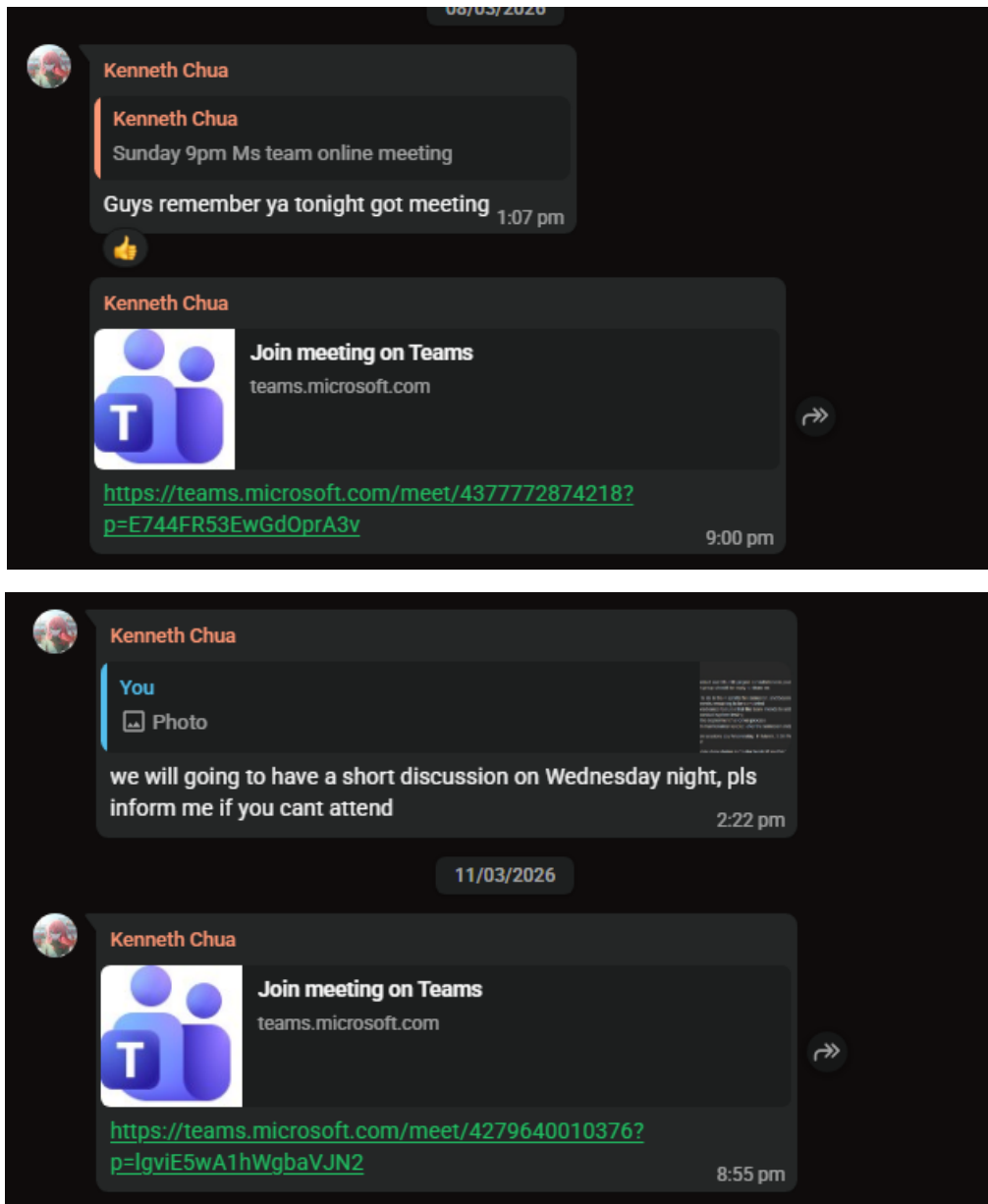


Figure 51: WhatsApp Chats

3.2.3 Microsoft Teams

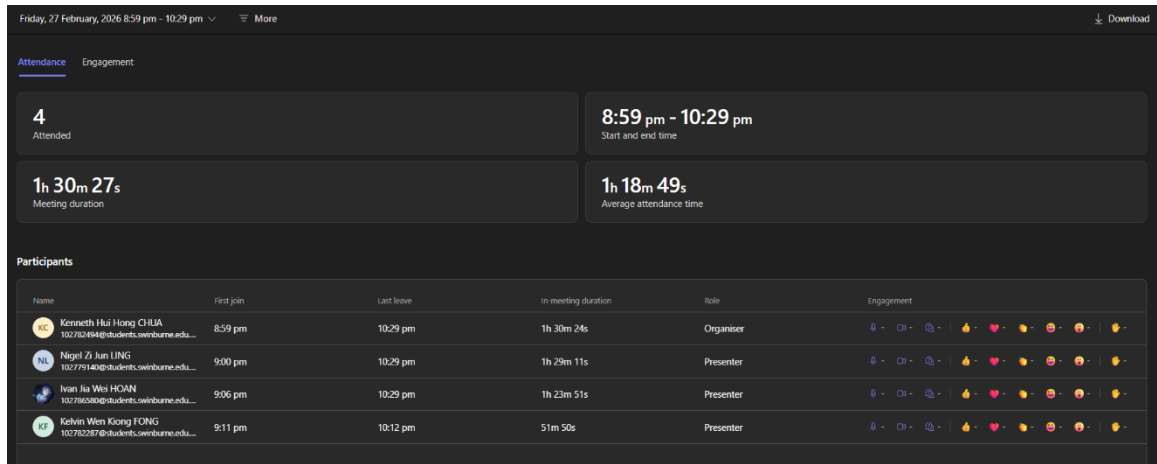


Figure 52: Meetings Channel

3.2.4 Linear

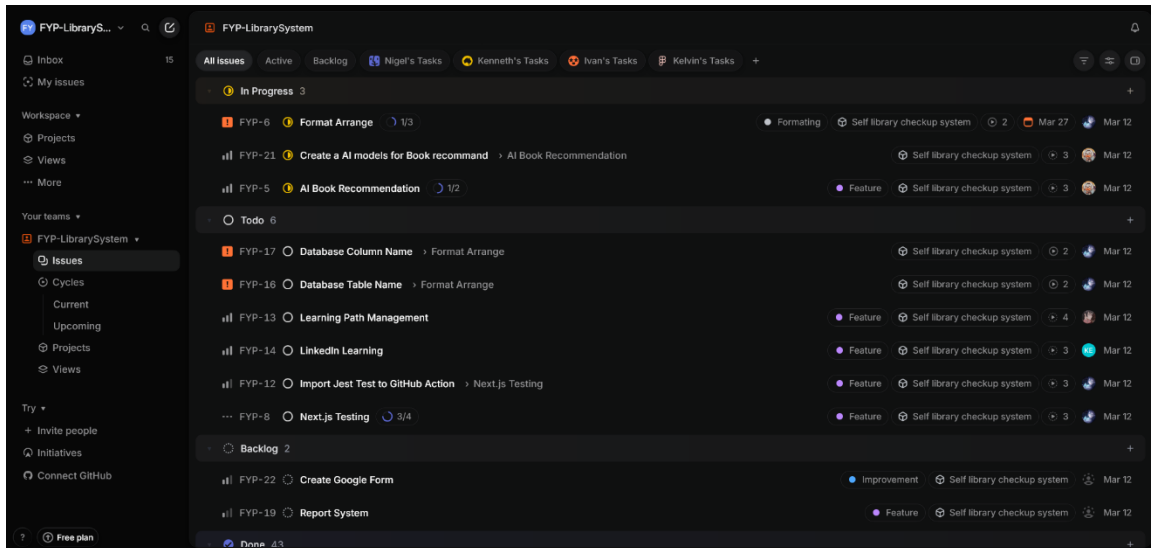


Figure 53: Linear Issue

This figure shows all the in progress, to-do, backing, and done issues in this sprint. Each issue is assigned to one or more members, with a priority level and current status, helping the team decide which task should be done or solved first. The key issues completed in this sprint included the Learning path basic implementation, Login page unit tests, and SIP2 credential security fix. Issues are categorized by type such as feature and improvement, allowing the team to track the breakdown of work done. Remain issues such as “AI model recommendation” and “LinkedIn Learning” will be carried over and prioritized in the next sprint cycle.

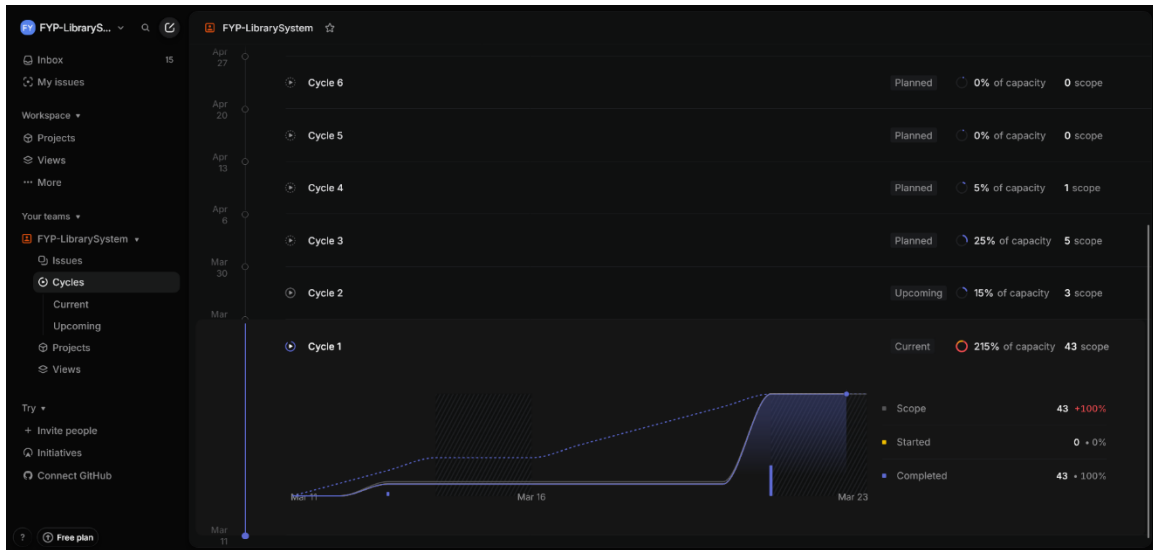


Figure 54: Linear Cycle

This figure shows the Sprint 2 cycle board in Linear. This sprint focused on implementing the enhancement features such as AI recommendations systems, learning path systems, and LinkedIn learning systems. The reason why there is an exponential growth around 20th of March is because the group members are not familiar with this (Linear) system and forgot to update the progress that has been completed days or weeks before. A total of 56 issues were planned for this cycle and 44 of them were completed. The incomplete issues such as learning path system were primarily caused by the Gemini API key being revoked in mid-sprint, which blocked full AI functionality from being tested and demonstrated. We expect to make more improvements to our current systems in the next cycle and plan to deploy them in the next sprint.

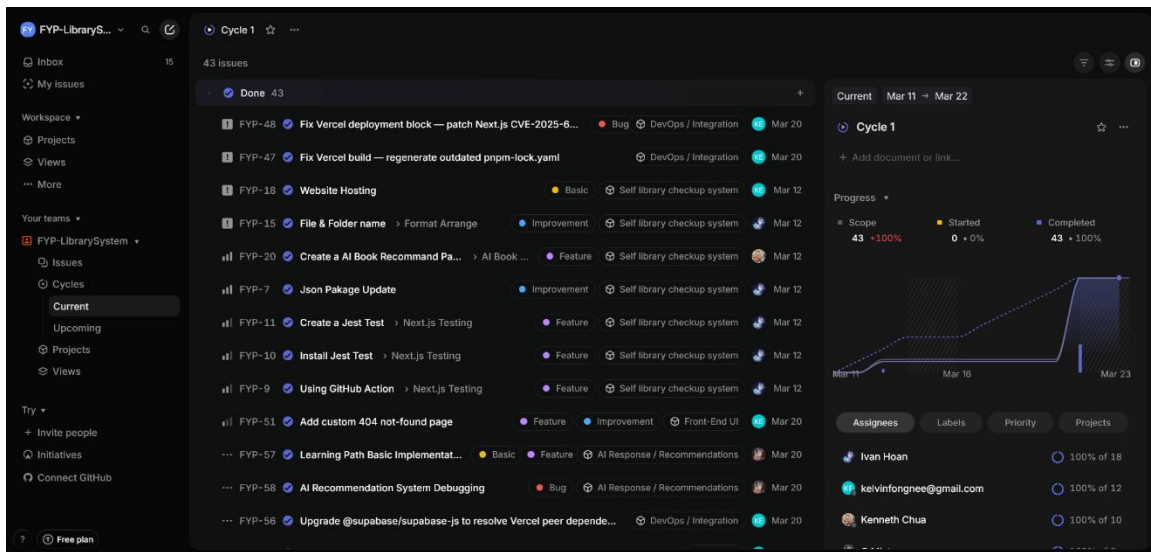


Figure 55: Linear Current Cycle

Figure 41 shows Cycle 1 of the Library Self-Check Out system project, running from March 11 to March 22. The cycle contained a total of 44 issues, all of which have been marked as Done, achieving 100% completion. The burndown chart on the right confirms that all 44 issues within the scope were completed by the end of the cycle. The issues are broken down by labels where 11 Feature issues (100% done), 7 Improvement issues (100% done), 3 Basic issues (100% done), and 3 Bug fixes (100% done). Key tasks completed this cycle include the AI Book Recommendation page, AI model integration, Learning Path basic implementation, AI Recommendation System debugging, website hosting, unit testing setup, and DevOps fixes such as patching the Next.js CVE-2025-66478 security vulnerability and resolving Supabase peer dependency warnings.

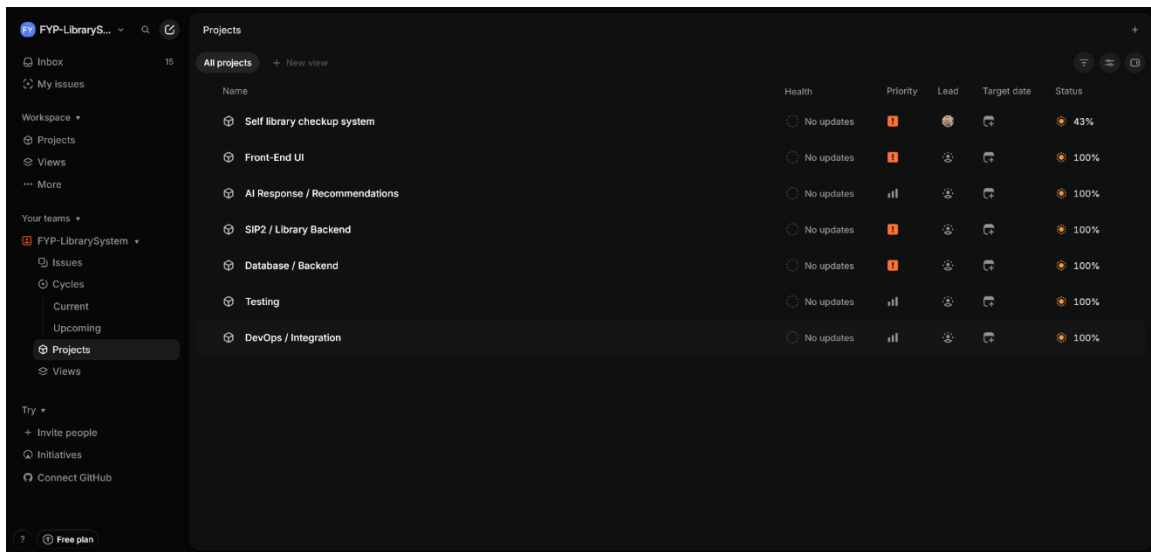


Figure 56: Linear Project

This figure shows the overall project board in Linear in Library Self-Checkout System, broken down into 7 sub-projects covering different areas of the system. Most subprojects including Front-End UI, AI Response/Recommendations, SIP2/Library Backend, Database/Backend, Testing, and DevOps/Integration have reached 100% completion. The main Self Library Checkup System project currently sits at 45% overall completion, reflecting that while the core infrastructure and individual modules are complete, higher-level and advanced integration and AI feature refinement are still ongoing. The orange priority indicators on Self Library Checkup System, Front-End UI, SIP2/Library Backend, and Database/Backend indicate these areas carry high priority within the project.

4 Sprint review

4.1 DEMONSTRATION SESSION AND CLIENT COMMUNICATION

Professional communication with the client, Mr. Nilson Fam, was conducted exclusively via Outlook email to maintain a formal record and respect the client's schedule. To provide a clear demonstration of progress without imposing on the client's time, comprehensive reports were prepared in Word and PDF formats. These documents featured screenshots of the system interface to provide visual evidence of developed features, including the AI recommendation chat and the learning path management system.

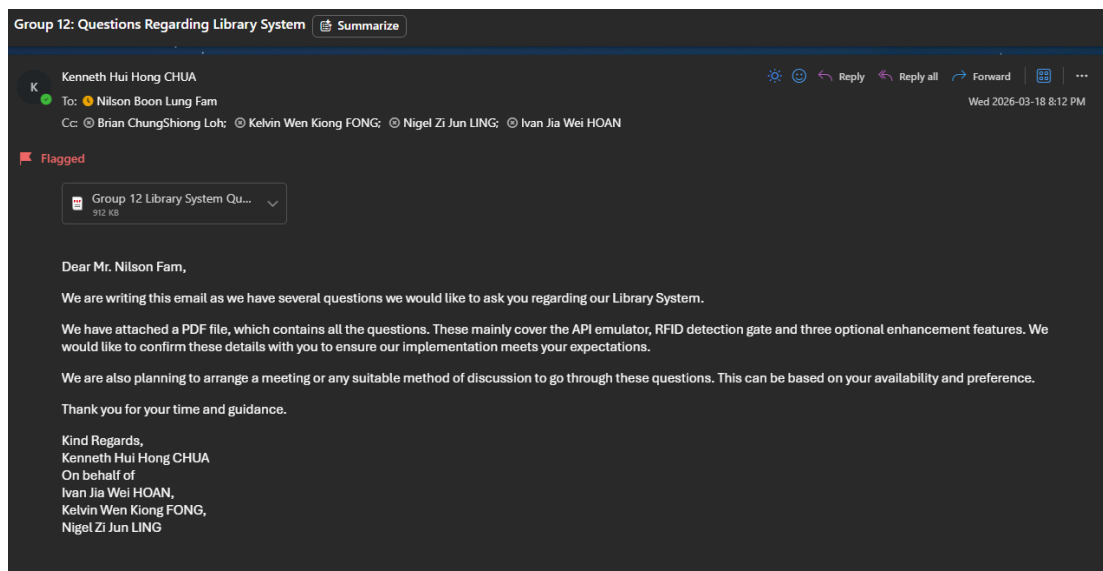


Figure 57: Email via Outlook for Client Questionnaire

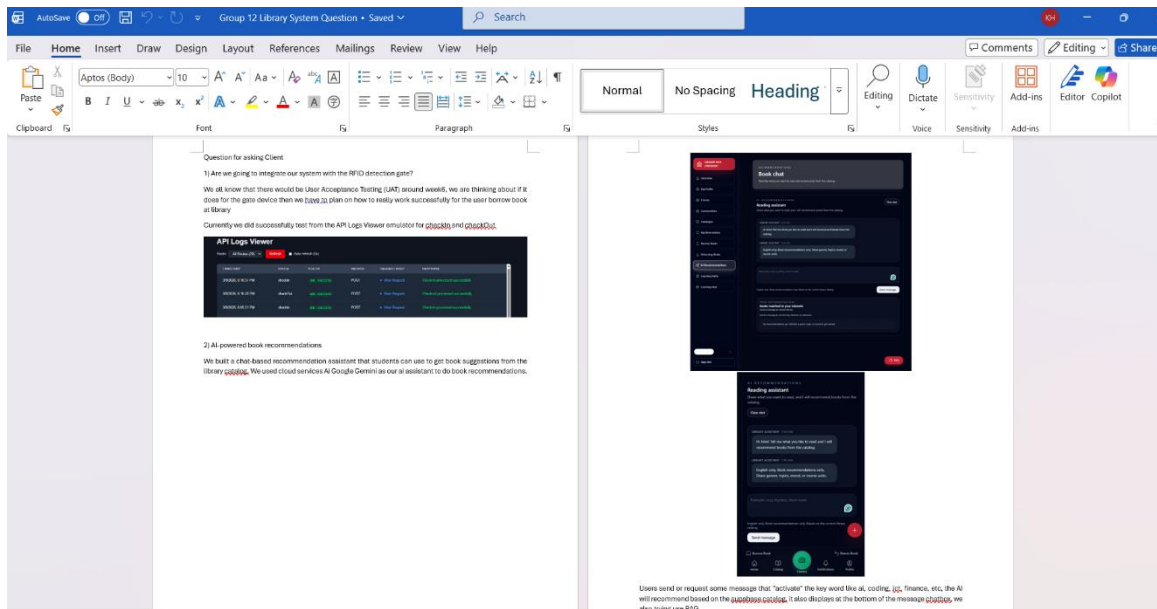


Figure 58: Example of questionnaire In MS. Word

Ongoing consultation with the Unit Convenor, Mr. Brian Loh, was conducted through physical meetings on campus at office E201. These on-site meetings allowed for direct discussion of project progress and helped ensure the implementation remains consistent with the academic requirements for Computing Technology Project B.

4.2 FEEDBACK RECEIVED

The development process has been significantly guided by technical feedback from the Unit Convenor, Mr. Brian Loh, during on-site consultations at office E201. A primary suggestion involved adopting a linear approach for the learning path management system, where the sequence of recommended readings is determined by both the type and the difficulty level of the books. This structural logic enables the generation of paths tailored to specific user needs, such as guiding a student from the basics of programming through to complex topics like data structures and web deployment. Further technical discussions addressed the potential deployment of on-premises Large Language Models (LLMs) to facilitate offline functionality, though it was noted that such an implementation would increase server storage requirements. Regarding the user interface, feedback highlighted the necessity of simplifying the navigation bar to reduce visual clutter and ensure a more intuitive experience for new user.

As of the current project status, no formal feedback has been received from the client, Mr. Nilson Fam, following the project status update sent via Outlook email.

4.3 CRITICAL ANALYSIS OF PROGRESS

Sprint 1 resulted in the successful transition from foundational architecture to the implementation of complex, AI-driven features. A primary technical milestone involved integrating Google Gemini 2.5 Flash into a Retrieval-Augmented Generation (RAG) pattern. This specific design ensures that book recommendations remain grounded in the live Supabase catalog rather than relying on general training data. Furthermore, technical agility was demonstrated through the identification and patching of a critical remote code execution vulnerability, CVE-2025-55182, which carried a maximum CVSS risk score of 10.0. Infrastructure stability was also enhanced by establishing a production deployment on Vercel and migrating sensitive SIP2 communications to the server-side to prevent credential exposure in browser logs.

Security hardening remained a focal point, as environment variables were successfully protected and sensitive keys were removed from network payloads. Testing infrastructure also saw significant progress, with the total suite reaching 28 passing unit tests with zero failures. These tests covered core user interface components, rendering, and accessibility, providing a stable safety net for future development cycles. Standardizing naming conventions to camelCase and migrating database table references to PascalCase further aligned the codebase with specific client requirements.

Several external hurdles and functional limitations impacted the overall project trajectory during this period. The planned LinkedIn Learning integration was deferred because enterprise API credentials have not yet been provisioned by the client's IT department. To mitigate this delay, a stub mode using sample data was developed to ensure the user interface logic remains functional once live credentials arrive. Additionally, the AI recommendation system currently struggles to process negative instructions and multi-topic queries, which often leads to diluted or irrelevant results. Data integrity within the Supabase database also presented challenges, as inconsistent book availability status frequently marks items as "On loan" despite no active loan records existing.

Project management analysis reveals that the Linear burndown chart experienced exponential growth toward the end of the sprint cycle. This surge occurred because task statuses were not updated in real-time as completion occurred. A mid-sprint interruption also occurred when the Gemini API key was revoked, which temporarily blocked full AI functionality and necessitated the use of a deterministic fallback mechanism. Refining prompt engineering and cleaning up stale database records are identified as high-priority tasks to ensure availability data becomes trustworthy in subsequent sprints.

5 Retrospect (Critical review of the process)

- **What went well**

Sprint 1 resulted in the resolution of major security vulnerabilities, which significantly enhanced the integrity of the website. An attractive and user-friendly interface was successfully implemented, facilitating an easier learning curve for new users. The system was successfully hosted and demonstrated operational stability in the current environment.

Website Link: <https://library-self-checkout-system-bh1b.vercel.app/>

Learning path generated successfully, the system correctly retrieves books from the library catalogue and groups them into Beginner, Intermediate, and Advanced stages. Also, the Fallback system works even when Gemini is unavailable, the system still generates a path instead of breaking and this system is Real-time available, book availability status is pulled live from Supabase and displays correctly.

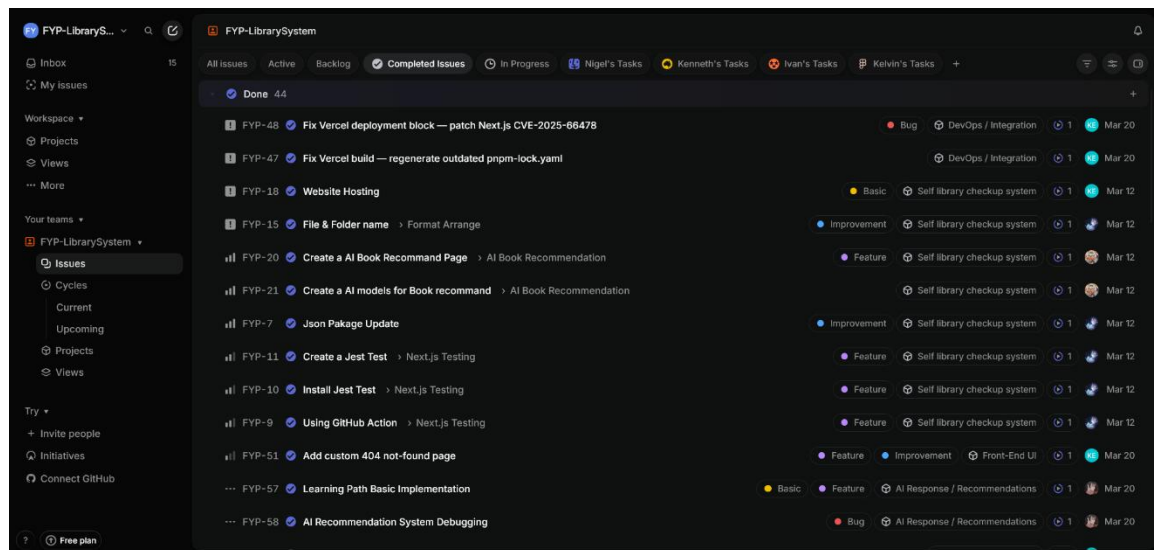


Figure 59: Completed Task

- **What did not go well**

The fine-tuning of the AI recommendation system presented significant challenges, as the current chatbot requires further refinement to increase the intelligence of responses. Although the system is functional, the project is currently behind the initial schedule, which has prevented immediate full-scale deployment.

For the learning path system, the book availability data is inconsistent, many books incorrectly show "On loan" in the database even though no active loan exists, making the availability feature unreliable. Also, the book reasons are not meaningful. Without Gemini working, every book shows "Recommended for this level" instead of a proper explanation.

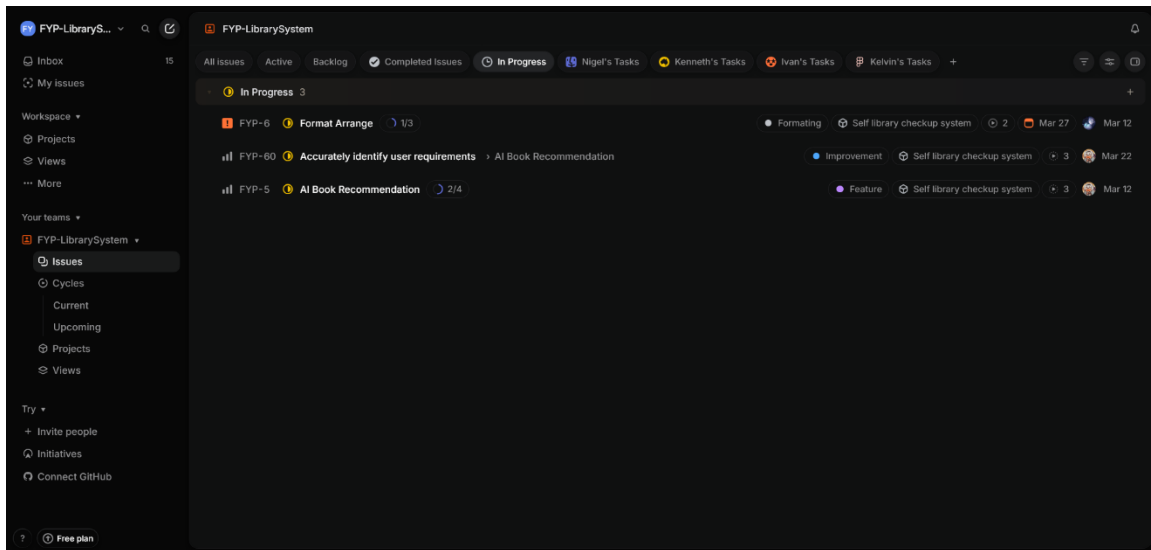


Figure 60: In Progress Task

- **What can be improved**

Improving the implementation of the AI model is necessary to address current deficiencies in recommendation quality. The system currently fails to accurately process negative instructions, such as queries containing "no" or "don't," leading to unsuitable book suggestions. Investigating more suitable and lower-cost AI alternatives is identified as a requirement for long-term project sustainability. While initial security gaps were closed, further enhancements are required to mitigate external hacking risks and improve the overall user experience. Resolving the Gemini API key issues is the single most critical improvement needed to unlock proper stage classification and meaningful explanations. Simultaneously, purging stale loan data from Supabase is essential to ensure that availability metrics are trustworthy.

- **Action item to be done**

Preparation for the upcoming testing phase is a priority, requiring the completion of all remaining core functions. A primary action item is the implementation of a notification mode to alert users of book returns, reservation availability, and staff-issued announcements. Finalizing the learning path management and LinkedIn Learning integration is also required for the next stage of development.

- **General observations from Sprint 1**

General observations indicate that the project is nearing readiness for the testing phase, provided the current development plan is adhered to. Success depends on the systematic resolution of identified problems and the management of critical risks. These risks include successful integration with the physical library system and managing testing limitations at the Swinburne library facility.

- **Risk Registry and Ethical Considerations**

The project remains dependent on the successful integration with the physical library system to ensure that core functionalities, such as book checkouts and returns, operate correctly. Heavy reliance on third-party services, specifically the Google Gemini API, introduces significant risks related to service instability or unexpected cost overruns. Furthermore, the implementation of the LinkedIn Learning feature is currently stalled due to the non-receipt of enterprise API credentials from the client's IT department, necessitating a deferral of this task to future sprints. To maintain functional continuity, a fallback algorithm was implemented for the learning path feature to ensure the system remains operational even if the AI service becomes unavailable.

Cybersecurity risks previously identified, such as the exposure of sensitive SIP2 credentials in browser logs and network requests, have been mitigated through the implementation of server-side handling and environment variable protection. Maintaining low risk levels in the future requires continuous monitoring of security advisories and regular dependency updates to protect against emerging vulnerabilities. Ongoing database schema migrations, involving the renaming of tables to PascalCase and columns to snake_case, carry the inherent risk of breaking existing queries unless migrations are conducted in multiple passes and verified meticulously.

Operational risks include potential progress delays due to uneven technical skills, which are currently being mitigated through direct monitoring and clearer task assignments. Testing limitations also present a challenge, as user acceptance testing for the RFID detection gate and overall system validation must be conducted physically at the Swinburne library. These site-specific constraints, combined with currently limited unit test coverage, necessitate a progressive approach to expanding the test suite to ensure all components are covered by automated validation.

Ethical considerations center on the rigorous protection of user privacy and the reliability of AI-generated content. Safeguarding student and staff data through centralized session validation and role-based access control is essential to prevent unauthorized access to restricted routes. Furthermore, the system must address ethical concerns regarding AI reliability by refining the Retrieval-Augmented Generation (RAG) pipeline to minimize inaccurate recommendations and ensure that explanations provided to users are meaningful and pedagogically sound. Maintaining data integrity is also a key ethical requirement, as inconsistent availability records must be corrected to prevent user frustration and ensure the system remains a trustworthy resource.

6 Lessons learned

(Critical review of SPRINT 1 experience and future plan)

Lesson Learned:

- Maintaining consistent communication with the client while simultaneously tracking internal progress is vital for aligning technical development with stakeholder needs.
- Continuous online research into documentation and emerging technologies is a fundamental requirement for identifying suitable software tools and plugins.
- Developing effective techniques for AI implementation remains a significant technical challenge, specifically in ensuring that large language models provide high-quality, relevant responses. Refining AI outputs from models such as GPT, Gemini, or Claude requires innovative, "out-of-the-box" strategies for processing varied user inputs.
- We were able to leverage the Gemini AI model in the implementation of automated book classification, which provides structured learning recommendations to users based on the difficulty level of the content: Beginner, Intermediate, and Advanced.
- As we worked on the project, we realized that there is a possibility that the AI model might not be stable in its response; hence, we implemented a fallback solution to ensure that there is a basic form of learning provided to the user even in the absence of AI services.
- The authentication process via OAuth 2.0 was successfully mastered, and an automated refresh mechanism was implemented successfully.
- Through the creation of stub data as we awaited our formal API credentials, we were able to continue development of our front-end interface, highlighting the need for parallel development.

To-do and future improvements:

- Perform continuous optimization of the system interface to enhance the user experience across both desktop and mobile platforms.
- Enhance website security protocols to safeguard internal data and ensure the preservation of user privacy.
- Achieve full implementation of the learning path system to provide structured educational progressions based on library resources.
- Another feature we plan to implement is a user progress tracking feature, which will enable users to save personal learning paths and mark resources as completed.
- We plan to continue optimizing the learning path algorithm by integrating it more closely with our library's real-time inventory system to ensure all recommended books are currently available.
- The next step would be to procure the production API credentials to facilitate the smooth transition from the stubbed data to the actual LinkedIn Learning courses' data.
- We plan to include an AI-driven federated search feature that would allow users to access physical books as well as relevant LinkedIn Learning video courses.